

```
1 % clear all
2 % close all
3 % clc
4
5 nRuns = 10000;
6 Mmax = 440; %número máximo de janela
7 alfa = 0.05;
8 FP_desejado =0.05;
9
10
11 [alfa_corrigido,NDC_minimo,cost_alfa, P] = funcao_NDC_alfaCorrigido_Mmax✓
(nRuns,Mmax ,alfa,FP_desejado);
12 %salvar as variáveis
13 save(['NDC_AlfaCorrigido_Mmax_Col' num2str(Mmax) '_alfa_' num2str(alfa)✓
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
14      'NDC_minimo','P', 'nRuns','alfa','FP_desejado')
15
16
17 t1 = table(P, NDC_minimo,alfa_corrigido,cost_alfa);
18
19 %% Avaliar a variância na estimação do NDC e alfa
20
21 %repetir isso várias vezes
22 % [alfa_corrigido,NDC_minimo,cost_alfa, P] =✓
funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax ,alfa_teste,FP_desejado);
23 % comparar os valor estimados e calcular a variância
24
25 %%
26
27
28
29
```

```
1 function [NDC,FP] = estimarNDC(NDCinicial, alfa, FP_desejado, ord, Mmin, Mstep, Mmax)
2
3
4
5 %parametros de entrada
6 % NDCinicial = 10;
7 % Mmin = 10;
8 % Mstep = 1;
9 % Mmax = 30;
10 %ord ->vetor com as ORDs - cada linha uma simulação
11
12
13 MM = Mmin:Mstep:Mmax;
14 NNTmax = size(MM,2); %número de teste
15 nRuns = size(ord,1);
16 fprintf('Mmin =%d - Mstep=%d - Mmax=%d - Nruns=%d -NDCinicial=%d\n', Mmin, Mstep, Mmax, nRuns, NDCinicial);
17
18 %calcular o falso positivo
19 dr = zeros(nRuns,1);
20 FP =0;
21 for ii = 1:nRuns
22     dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDCinicial);
23 end
24 FP = mean(dr);
25
26 if (FP<FP_desejado)
27     if NDCinicial>3 %%mudei 1 --> 3
28         warning('NDC inicial grande --- Isso pode aumentar o tempo do código');
29         NDCinicial = 1;
30     end
31 end
32
33 FP = 0;
34 for NDC = NDCinicial:NNTmax %número máximo de teste
35
36     for ii = 1:nRuns
37         dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDC);
38     end
39     FP(NDC) = mean(dr);
40
41     if (FP(NDC)<FP_desejado)
42         break;
43     end
44
45 end
```

```
46
47
48 %encontrar o valor mínimo
49 [~,ind] = find((FP<FP_desejado).*(FP~=0)); %encontrar o primeiro NT que✓
%respeita a restrição
50
51 NDC = 0;
52 if isempty(ind)
53     NDC = nan;
54     warning('##### Não encontro Nalpha #####');
55     elseif ind(1)>1
56         NDC = interp1(FP([ind(1)-1,ind(1)]),[ind(1)-1,ind(1)],✓
FP_desejado);
57     else
58         NDC = 1; %para apenas uma detecção
59 end
60
61 if(NNTmax ==2)
62     NDC=2;
63 end
64
65 if(NNTmax ==1)
66     NDC=1;
67 end
68
69
```

```
1 function [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
2 % Minimize a continuous differentiable multivariate function. Starting
point
3 % is given by "X" (D by 1), and the function named in the string "f",
must
4 % return a function value and a vector of partial derivatives. The
Polack-
5 % Ribiere flavour of conjugate gradients is used to compute search
directions,
6 % and a line search using quadratic and cubic polynomial approximations
and the
7 % Wolfe-Powell stopping criteria is used together with the slope ratio
method
8 % for guessing initial step sizes. Additionally a bunch of checks are
made to
9 % make sure that exploration is taking place and that extrapolation
will not
10 % be unboundedly large. The "length" gives the length of the run: if it
is
11 % positive, it gives the maximum number of line searches, if negative
its
12 % absolute gives the maximum allowed number of function evaluations.
You can
13 % (optionally) give "length" a second component, which will indicate
the
14 % reduction in function value to be expected in the first line-search
(defaults
15 % to 1.0). The function returns when either its length is up, or if no
further
16 % progress can be made (ie, we are at a minimum, or so close that due
to
17 % numerical problems, we cannot get any closer). If the function
terminates
18 % within a few iterations, it could be an indication that the function
value
19 % and derivatives are not consistent (ie, there may be a bug in the
20 % implementation of your "f" function). The function returns the found
21 % solution "X", a vector of function values "fX" indicating the
progress made
22 % and "i" the number of iterations (line searches or function
evaluations,
23 % depending on the sign of "length") used.
24 %
25 % Usage: [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
26 %
27 % See also: checkgrad
28 %
29 % Copyright (C) 2001 and 2002 by Carl Edward Rasmussen. Date 2002-02-13
```

```
30 %
31 %
32 % (C) Copyright 1999, 2000 & 2001, Carl Edward Rasmussen
33 %
34 % Permission is granted for anyone to copy, use, or modify these
35 % programs and accompanying documents for purposes of research or
36 % education, provided this copyright notice is retained, and note is
37 % made of any changes that have been made.
38 %
39 % These programs and documents are distributed without any warranty,
40 % express or implied. As the programs were written for research
41 % purposes only, they have not been tested to the degree that would be
42 % advisable in any important application. All use of these programs is
43 % entirely at the user's own risk.
44 %
45 % [ml-class] Changes Made:
46 % 1) Function name and argument specifications
47 % 2) Output display
48 %
49
50 % Read options
51 if exist('options', 'var') && ~isempty(options) && isfield(options, ✓
'MaxIter')
52     length = options.MaxIter;
53 else
54     length = 100;
55 end
56
57
58 RHO = 0.01; % a bunch of constants for line ✓
searches
59 SIG = 0.5; % RHO and SIG are the constants in the Wolfe-Powell ✓
conditions
60 INT = 0.1; % don't reevaluate within 0.1 of the limit of the current ✓
bracket
61 EXT = 3.0; % extrapolate maximum 3 times the current ✓
bracket
62 MAX = 20; % max 20 function evaluations per ✓
line search
63 RATIO = 100; % maximum allowed ✓
slope ratio
64
65 argstr = ['feval(f, X']; % compose string used to ✓
call function
66 for i = 1:(nargin - 3)
67     argstr = [argstr, ',P', int2str(i)];
68 end
69 argstr = [argstr, ')]';
```

```

70
71 if max(size(length)) == 2, red=length(2); length=length(1); else red=1;✓
end
72 S=['Iteration '];
73
74 i = 0; % zero the run length✓
counter
75 ls_failed = 0; % no previous line search✓
has failed
76 fX = [];
77 [f1 df1] = eval(argstr); % get function value and✓
gradient
78 i = i + (length<0); % count✓
epochs?!
79 s = -df1; % search direction is✓
steepest
80 d1 = -s'*s; % this is✓
the slope
81 z1 = red/(1-d1); % initial step is✓
red/(|s|+1)
82
83 while i < abs(length) % while not✓
finished
84 i = i + (length>0); % count✓
iterations?!
85
86 X0 = X; f0 = f1; df0 = df1; % make a copy of✓
current values
87 X = X + z1*s; % begin✓
line search
88 [f2 df2] = eval(argstr);
89 i = i + (length<0); % count✓
epochs?!
90 d2 = df2'*s;
91 f3 = f1; d3 = d1; z3 = -z1; % initialize point 3 equal to✓
point 1
92 if length>0, M = MAX; else M = min(MAX, -length-i); end
93 success = 0; limit = -1; % initialize quantities
94 while 1
95 while ((f2 > f1+z1*RHO*d1) || (d2 > -SIG*d1)) && (M > 0)
96 limit = z1; % tighten the✓
bracket
97 if f2 > f1
98 z2 = z3 - (0.5*d3*z3*z3)/(d3*z3+f2-f3); %✓
quadratic fit
99 else
100 A = 6*(f2-f3)/z3+3*(d2+d3); %✓
cubic fit

```

```

101      B = 3*(f3-f2)-z3*(d3+2*d2);
102      z2 = (sqrt(B*B-A*d2*z3*z3)-B)/A;           % numerical error✓
possible - ok!
103      end
104      if isnan(z2) || isinf(z2)
105          z2 = z3/2;                             % if we had a numerical problem✓
then bisect
106      end
107      z2 = max(min(z2, INT*z3), (1-INT)*z3); % don't accept too close✓
to limits
108      z1 = z1 + z2;                               % update✓
the step
109      X = X + z2*s;
110      [f2 df2] = eval(argstr);
111      M = M - 1; i = i + (length<0);              % count✓
epochs?!
112      d2 = df2'*s;
113      z3 = z3-z2;                                % z3 is now relative to the✓
location of z2
114      end
115      if f2 > f1+z1*RHO*d1 || d2 > -SIG*d1
116          break;                                  % this is a✓
failure
117      elseif d2 > SIG*d1
118          success = 1; break;                      %✓
success
119      elseif M == 0
120          break;                                   %✓
failure
121      end
122      A = 6*(f2-f3)/z3+3*(d2+d3);                  % make cubic✓
extrapolation
123      B = 3*(f3-f2)-z3*(d3+2*d2);
124      z2 = -d2*z3*z3/(B+sqrt(B*B-A*d2*z3*z3));    % num. error✓
possible - ok!
125      if ~isreal(z2) || isnan(z2) || isinf(z2) || z2 < 0 % num prob or✓
wrong sign?
126          if limit < -0.5                          % if we have no✓
upper limit
127              z2 = z1 * (EXT-1);                    % the extrapolate the✓
maximum amount
128          else
129              z2 = (limit-z1)/2;                      %✓
otherwise bisect
130          end
131          elseif (limit > -0.5) && (z2+z1 > limit)    % extrapolation✓
beyond max?
132          z2 = (limit-z1)/2;✓

```





```
give up
164     end
165     tmp = df1; df1 = df2; df2 = tmp;                % swap✓
derivatives
166     s = -df1;                                       % try✓
steepest
167     d1 = -s'*s;
168     z1 = 1/(1-d1);
169     ls_failed = 1;                                % this line✓
search failed
170     end
171     if exist('OCTAVE_VERSION')
172         fflush(stdout);
173     end
174 end
175 fprintf('\n');
176
```

```
1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido,cost_alfa, P] =funcao_alfaCorrigido_Mmax(nRuns, ✓
Mmax,FP_desejado)
6 %parametros de entrada -----
7 % pentrada = ;
8 FP_desejado = 0.05;
9 nRuns = 10;
10 Mmax = 440; %número máximo de janela
11 %-----
12 %parâmetros default
13 % fs = 64;
14 tj = 32; %cada janela um segundo
15 bin = 8;
16
17 Ntotal = Mmax*tj; %número de pontos totais
18
19 %Na simulação iremos estimar a aplicação do detector a cada janela
20 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
21
22 for ii = 1: nRuns
23     x = randn(Ntotal,1);
24     x = reshape(x,tj,Mmax); %dividir em janelas
25     %aplicar o detector a cada janela -----
26     xfft = fft(x); %aplico uma única vez a FFT.
27     for M = 2:Mmax %fazer para cada acrescimo de uma janela
28         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
29     end
30 end
31
32
33 %% obter todos os parâmetros
34 P = parametros_protocolo(Mmax);
35 % P=pentrada;
36 alfa_corrigido = nan*ones(size(P,1),1);
37 cost_alfa = nan*ones(size(P,1),1);
38
39 for ii = 1:size(P,1)
40     Mmin = P(ii,1);
41     Mstep = P(ii,2);
42     Mmax = P(ii,3);
43     MM = Mmin:Mstep:Mmax;
44     disp([num2str(ii*100/size(P,1)),'%'])
45
46     det = ord(:,MM);
```

```
47     alfa = 0.05; %TAXA DE FALSO POSITIVO DE CADA TESTES
48     options = optimset('MaxIter', 50);
49     cc = @(alfa) funcao_custo(alfa ,MM, det, FP_desejado);
50     [alfa, cost] = fmincg(cc,alfa, options);
51     alfa_corrigido(ii) = alfa;
52
53     if ~isempty(cost)
54         cost_alfa(ii) = cost(end);
55     end
56
57 end
58 %salvar as variáveis
59 save(['NDC_Unitario_Mmax_' num2str(Mmax) '_alfa_' num2str(alfa) ✓
'_FPdesejado' num2str(FP_desejado) '.mat'], 'alfa_corrigido', ...
60     'NDC_minimo', 'P', 'nRuns', 'alfa', 'FP_desejado')
61
62 t1 = table(P, NDC_minimo, alfa_corrigido, cost_alfa);
63 % figure
64 % plot(alfa_corrigido, 'ok')
65 % figure
66 % plot(cost_alfa, 'ok')
67 %ylim([min(cost_alfa) max(cost_alfa)])
68
69
70
71
72
73
```

```
1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido,cost_alfa, P] =funcao_alfaCorrigido_Mmax(nRuns, ✓
Mmax,FP_desejado)
6 %parametros de entrada -----
7 % pentrada = ;
8 FP_desejado = 0.05;
9 nRuns = 100;
10 Mmax = 20; %número máximo de janela
11 alfa_teste = 0.05;
12 %-----
13 %parâmetros default
14 % fs = 64;
15 tj = 32; %cada janela um segundo
16 bin = 8;
17
18 Ntotal = Mmax*tj; %número de pontos totais
19
20 %Na simulação iremos estimar a aplicação do detector a cada janela
21 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
22
23 for ii = 1: nRuns
24     x = randn(Ntotal,1);
25     x = reshape(x,tj,Mmax); %dividir em janelas
26     %aplicar o detector a cada janela -----
27     xfft = fft(x); %aplico uma única vez a FFT.
28     for M = 2:Mmax %fazer para cada acrescimo de uma janela
29         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
30     end
31 end
32
33
34 %% obter todos os parâmetros
35 P = parametros_protocolo(Mmax);
36 % P=pentrada;
37 alfa_corrigido = nan*ones(size(P,1),1);
38 cost_alfa = nan*ones(size(P,1),1);
39
40 for ii = 1:size(P,1)
41     Mmin = P(ii,1);
42     Mstep = P(ii,2);
43     Mmax = P(ii,3);
44     MM = Mmin:Mstep:Mmax;
45     disp([num2str(ii*100/size(P,1)),'%'])
46
```

```
47     det = ord(:,MM);
48     alfa = 0.05; %TAXA DE FALSO POSITIVO DE CADA TESTES
49     options = optimset('MaxIter', 50);
50     cc = @(alfa) funcao_custo(alfa ,MM, det, FP_desejado);
51     [alfa, cost] = fmincg(cc,alfa, options);
52     alfa_corrigido(ii) = alfa;
53
54     if ~isempty(cost)
55         cost_alfa(ii) = cost(end);
56     end
57
58 end
59
60 %salvar as variáveis
61 alfa = alfa_teste; % eu coloquei
62 save(['NDC_Unitario_Mmax_' num2str(Mmax) '_alfa_' num2str(alfa) ✓
63     '_FPdesejado' num2str(FP_desejado) '.mat'], 'alfa_corrigido', ...
64     'P', 'nRuns', 'alfa', 'FP_desejado')
65
66 t1 = table(P, alfa_corrigido, cost_alfa);
67 % figure
68 % plot(alfa_corrigido, 'ok')
69 % figure
70 % plot(cost_alfa, 'ok')
71 %ylim([min(cost_alfa) max(cost_alfa)])
72
73
74
75
76
```

```
1 function [J,grad] = funcao_custo(alfa,MM,det, FP_desejado)
2
3 nRuns = size(det,1);
4 %1-Critério de parada -----
5 valor_critico = VC_MSC(MM,alfa);
6 det = det>repmat(valor_critico,nRuns,1);
7 FP = mean((sum(det,2)>0)); %TAXA DE FALSO POSITIVO DA SEÇÃO (EXAME)
8
9 %2 - Cálculo da função de custo -----
10 erro = (FP-FP_desejado);
11 %J = erro.^2;
12 J = erro.^2+100*(alfa<0);
13
14
15 %3 - Cálculo do gradiente -----
16 %grad = erro*FP;
17 grad = erro*FP; %~porporcional ao gradiente (não é o verdadeiro - mas✓
porporcional)
18
19
```

```
1 function [J,grad] = funcao_custo_v2(alfa, NDC, MM, ord, FP_desejado)
2
3 nRuns = size(ord,1);
4 alfa = max(alfa,eps);
5 alfa = min(alfa,1-eps);
6 %1-Critério de parada -----
7 dr = zeros(nRuns,1);
8 for ii = 1:nRuns
9     dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDC);
10 end
11 FP = mean(dr);
12
13 %2 - Cálculo da função de custo -----
14 erro = (FP-FP_desejado);
15 %J = erro.^2;
16 J = 1/2*erro.^2;
17
18
19 %3 - Cálculo do gradiente -----
20 grad = erro;
21
22 %grad = erro*FP; %~porporcional ao gradiente (não é o verdadeiro - mas ✓
porporcional)
23
```

```

1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido,NDC_minimo,cost_alfa, P] ✓
=funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax,alfa_teste,FP_desejado)
6 %parametros de entrada -----
7 % clear all, close all, clc
8 nRuns = 1000;
9 Mmax = 20; %número máximo de janela
10 alfa_teste = 0.05;
11 FP_desejado =0.05;
12 %-----
13
14 %parâmetros default
15 % fs = 64;
16 tj = 32; %cada janela um segundo
17 bin = 8;
18
19
20 Ntotal = Mmax*tj; %número de pontos totais
21
22 %Na simulação iremos estimar a aplicação do detector a cada janela
23 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
24
25 for ii = 1: nRuns
26     x = randn(Ntotal,1);
27     x = reshape(x,tj,Mmax); %dividir em janelas
28     %aplicar o detector a cada janela -----
29     xfft = fft(x); %aplico uma única vez a FFT.
30     for M = 2:Mmax %fazer para cada acrescimo de uma janela
31         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
32     end
33 end
34
35
36 %% obter todos os parâmetros
37 P = parametros_protocolo(Mmax);
38 alfa_corrigido = nan*ones(size(P,1),1);
39 cost_alfa = nan*ones(size(P,1),1);
40
41 NDC_minimo = 3+nan*ones(size(P,1),1); %NDC_mínimo
42
43 %%
44 [~,aux] = sort(ceil((Mmax - P(:,1))./P(:,2))+1);
45 P = P(aux,:);
46

```



```

47
48 for ii = 1:size(P,1)
49     Mmin = P(ii,1);
50     Mstep = P(ii,2);
51     Mmax = P(ii,3);
52     MM = Mmin:Mstep:Mmax;
53     disp([num2str(ii*100/size(P,1)),'%'])
54
55 %         %      NDC:sugerir um bom
56 if ii == 1
57     Ninicial = 1;
58 else
59     Ninicial = max(round(NDC_minimo(ii-1))-5,1); %% Mudei 3--> 5
60 end
61
62
63 [NDC,FP] = estimarNDC(Ninicial,alfa_teste,FP_desejado, ord, Mmin,✓
Mstep, Mmax);
64 NDC_minimo(ii) = NDC;
65 %     NDC_minimo(ii) = 1;
66
67 %ajustar os valores crítico
68 MM = Mmin:Mstep:Mmax;
69 options = optimset('MaxIter', 50);
70 cc = @(alfa) funcao_custo_v2(alfa, NDC_minimo(ii), MM, ord,✓
FP_desejado);
71 [alfa, cost] = fmincg(cc,alfa_teste, options);
72 alfa_corrigido(ii) = alfa;
73
74 %     if isempty(cost)
75 %         y= 1;
76 %     end
77
78 if ~isempty(cost)
79     cost_alfa(ii) = cost(end);
80 end
81
82
83 end
84
85 % alfa = alfa_teste;
86
87 % save('NNDC_AlfaCorrigido_Mmax20_alfa_0.05_FPdesejado0.05.✓
mat','alfa','alfa_corrigido','FP_desejado', 'NDC_minimo', 'P', 'nRuns');
88 % save(['NNDC_AlfaCorrigido_Mmax_' num2str(Mmax) '_alfa_' num2str✓
(alfa_teste) '_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido',✓
...
89 %     'P', 'nRuns','alfa','FP_desejado')

```

```
90 % figure
91 % plot(alfa_corrigido,'ok')
92 % figure
93 % plot(cost_alfa,'ok')
94 % ylim([min(cost_alfa) max(cost_alfa)])
95
96
97
98
99
100
```

```
1
2 %%
3 clearvars
4 linha = [line(),line(),line(),line()];
5 close all
6 clc
7
8 %%
9 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
10
11 cor = ['m','r','g','b'];
12
13 for metodo = 1:4
14     caminho1 = ['timeM_30db_M',num2str(metodo),'.mat'];
15     caminho2 = ['pareto_30db_M',num2str(metodo),'.mat'];
16
17     load(caminho1,'timeM')
18     load(caminho2,'TXD','Mmax','parametros');
19
20     % pontos e linhas pretos:
21     plot([0 1]*100, [Mmax Mmax], '-.k','linewidth',1)
22     plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k','linewidth',1)
23
24
25     for ii = 1:size(parametros,1)
26         plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',
27             [num2str(parametros(ii,1)) '-' num2str(parametros(ii,2))])
28     end
29
30     [p, idxs] = paretoFront([TXD,(-timeM)]);
31     auxL = p(:,1)<0.5;
32     p(auxL,:) = [];
33     idxs(auxL,:) = [];
34
35     [~,ind] = sort(p(:,1));
36     p = p(ind,:);
37     idxs = idxs(ind,:);
38
39     % Linha colorida:
40     tipo = ['-o' cor(metodo)];
41     linha(metodo) = plot(TXD(idxs),timeM(idxs),tipo,'Markersize',
42         8,'linewidth',1.8);
43
44     for ii = 1:size(idxs,1)
45         [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
46         I = I(1);
```

```
46
47     for jj = I
48         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(
49 (jj,1)) ',' num2str(parametros(jj,2)) '\}' ]);
50     end
51 end
52
53 %%
54 set(axes1,'XMinorTick','on');
55 set(axes1,'YMinorTick','on');
56 box(axes1,'off');
57
58
59 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
60 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
61
62 grid on
63 hold on
64 title('Curva de Pareto - INTENSIDADE 30 dB SPL')
65 legend([linha(1),linha(2),linha(3),linha(4)],'Método 1','Método 2',
66 'Método 3','Método 4')
67 xlabel('Taxa de detecção [%]')
68 ylabel('Tempo de exame [s]')
```

```
1 function P = parametros_protocolo(Mmax)
2
3 %função para obter todos os parâmetros possíveis de um protocolo de
4 %detecção a partir de um Mmax de entrada
5
6 %entrada: Mmax
7
8 %saída: Matriz P onde:
9         %cada linha indica um parâmetro.
10        %Coluna [ Min, Mstep, Mmax];
11
12 %%Código -----
13
14 P = [];
15
16 for M_step = 1:(Mmax-1)
17     for Mmin = 2:(Mmax-1)%MMentrada
18         k = (Mmax -Mmin)/M_step; %NTmax:nº max de teste ORD no✓
19         if (ceil(k) == floor(k) & k>=0) %verificar se o valor de k é um✓
20             P = [P;Mmin,M_step,Mmax];
21         end
22     end
23 end
24
25 %Incluir o teste único também
26 P = [P;Mmax 1 Mmax];
27
28
29
30
31
```

```

1 %%
2 clearvars
3 linha = [line(),line(),line(),line()];
4 close all
5 clc
6
7 %%
8 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
9 Intensidade = {'30dB'};
10 % Intensidade = {'50dB'};
11 cor = {'m','r','g','b'};
12
13 for metodo = 1:4
14     caminho1 = ['timeM_50db_M',num2str(metodo),'.mat'];
15     caminho2 = ['pareto_50db_M',num2str(metodo), '.mat'];
16 %     caminho1 = ['timeM_30db_M',num2str(metodo),'.mat'];
17 %     caminho2 = ['pareto_30db_M',num2str(metodo), '.mat'];
18
19     load(caminho1,'timeM')
20     load(caminho2,'TXD','Mmax','parametros');
21
22     % pontos e linhas pretos: SINGLE SHOT 50dB
23     single = plot([0 1]*100, [Mmax Mmax], '-.b','linewidth',1);
24 %     plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b','linewidth',1)
25 % SINGLE SHOT 30dB
26     plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.b','linewidth',1);
27
28
29
30     for ii = 1:size(parametros,1)
31         plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',
[num2str(parametros(ii,1)) '-' num2str(parametros(ii,2))])
32     end
33
34     [p, idxs] = paretoFront([TXD,(-timeM)] );
35     auxL = p(:,1)<0.5;
36     p(auxL,:) = [];
37     idxs(auxL,:) = [];
38
39     [~,ind] = sort(p(:,1));
40     p = p(ind,:);
41     idxs = idxs(ind,:);
42
43     % Linha colorida:
44     tipo = ['-o' cor(metodo)];
45     linha(metodo) = plot(TXD(idxs),timeM(idxs),tipo,'Markersize',
8,'linewidth',1.8) ;

```

```
46
47     for ii = 1:size(idxs,1)
48
49         [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
50         I = I(1);
51
52         for jj = I
53             text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros✓
(jj,1)) ',' num2str(parametros(jj,2)) '\}' ]);
54         end
55     end
56 end
57
58 %%
59 set(axes1,'XMinorTick','on');
60 set(axes1,'YMinorTick','on');
61 box(axes1,'off');
62
63 %
64 % xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
65 % ylim([min(timeM(idxs))*0.95 Mmax*1.05])
66
67 grid on
68 hold on
69 % title(['Curva de Pareto Intensidade ', (Intensidade{1}), ' SPL:✓
Métodos 01 a 04'])
70 title(['Curva de Pareto Intensidade ', (Intensidade{1}), ' SPL: Métodos✓
NND'])
71 % title('Curva de Pareto - 30 dB')
72
73 % legend([linha(1),linha(2),linha(3),linha(4), single],'Método✓
1','Método 2','Método 3','Método 4','Single test','Location','southeast')
74 legend([linha(1),linha(2),linha(3),linha(4), single],'NND = 1','NND =✓
2','NND = 3','NND = 5','Single test','Location','southeast')
75
76 xlabel('Taxa de detecção [%]')
77 ylabel('Tempo de exame [s]')
78
```

```
1 function [ p, idxs] = paretoFront( p )
2 % Filters a set of points P according to Pareto dominance, i.e., points
3 % that are dominated (both weakly and strongly) are filtered.
4 %
5 % Inputs:
6 % - P      : N-by-D matrix, where N is the number of points and D is the
7 %            number of elements (objectives) of each point.
8 %
9 % Outputs:
10 % - P      : Pareto-filtered P
11 % - idxs   : indices of the non-dominated solutions
12 %
13 % Example:
14 % p = [1 1 1; 2 0 1; 2 -1 1; 1, 1, 0];
15 % [f, idxs] = paretoFront(p)
16 %      f = [1 1 1; 2 0 1]
17 %      idxs = [1; 2]
18
19 [i, dim] = size(p);
20 idxs = [1 : i]';
21 while i >= 1
22     old_size = size(p,1);
23     indices = sum( bsxfun( @ge, p(i,:), p ), 2 ) == dim;
24     indices(i) = false;
25     p(indices,:) = [];
26     idxs(indices) = [];
27     i = i - 1 - (old_size - size(p,1)) + sum(indices(i:end));
28 end
29
30 end
```



```
1 function [dr,time] = protocolo_deteccao(x, parametros)
2 % mmaxx = 200*ones(size (parametros));
3 binsM = 120;
4 %c  cular o valor do detector
5 %aplicar o detector a cada janela -----
6 xfft = fft(x); %aplico uma   nica vez a FFT.
7
8 for M = 2:size(x,2) %fazer para cada acrescimo de uma janela
9     % ord(:,M) = msc_fft(xfft(bin,1:M),M);
10    ord(:,M) = msc_fft(xfft(1:binsM,1:M),M);
11 end
12
13
14 for ii = 1:size(parametros,1)
15
16     %parametros
17     Mmin = parametros(ii,1);
18     Mstep = parametros(ii,2);
19     Mmax = parametros(ii,3);
20     NDC = parametros(ii,4);
21     alfa = parametros(ii,5);
22
23     % protocolo
24     MM = Mmin:Mstep:Mmax;
25
26     if M <240
27         disp('aha')
28     end
29
30     for ll = 1:size(ord,1)
31
32         [dr(ll,ii),time(ll,ii)] = ETS(ord(ll,:),MM',alfa,NDC);
33     end
34
35 %     det = ord(:,MM);
36 %
37 %     valor_critico = VC_MSC(MM,alfa);
38 %     det = det>repmat(valor_critico,size(det,1),1);
39 %
40 %     %dr = double(sum(det,2)>1);
41 %     for ll = 1:size(ord,1)
42 %
43 %         dr(ll,ii) = double(sum(det(ll,:),2)>0);
44 %         if dr(ll,ii) ==0
45 %             time(ll,ii) = -1;
46 %         else
47 %             aux = find(det(ll,')==1);
48 %             time(ll,ii) = MM(aux(1));
```

```
49 %         end
50 %
51 %     end
52
53 end
54
55 % %% Não detecção
56 % function [dr,time] = protocolo_deteccao(x, parametros)
57 %
58 % binsM = 120;
59 % %cálculo o valor do detector
60 % %aplicar o detector a cada janela -----
61 % xfft = fft(x); %aplico uma única vez a FFT.
62 %
63 % for M = 2:size(x,2) %fazer para cada acrescimo de uma janela
64 %     % ord(:,M) = msc_fft(xfft(bin,1:M),M);
65 %     ord(:,M) = msc_fft(xfft(1:binsM,1:M),M);
66 % end
67 %
68 %
69 % for ii = 1:size(parametros,1)
70 %
71 %     %parametros
72 %     Mmin = parametros(ii,1);
73 %     Mstep = parametros(ii,2);
74 %     Mmax = parametros(ii,3);
75 %     NDC = parametros(ii,4);
76 %     alfa = parametros(ii,5);
77 %
78 %     % protocolo
79 %     MM = Mmin:Mstep:Mmax;
80 %
81 %     for ll = 1:size(ord,1)
82 %
83 %         [dr(ll,ii),time(ll,ii)] = ETS(ord(ll,:),MM',alfa,NDC);
84 %     end
85 %
86 % %     det = ord(:,MM);
87 % %
88 % %     valor_critico = VC_MSC(MM,alfa);
89 % %     det = det>repmat(valor_critico,size(det,1),1);
90 % %
91 % % %     %dr = double(sum(det,2)>1);
92 % %     for ll = 1:size(ord,1)
93 % %
94 % %         dr(ll,ii) = double(sum(det(ll,:),2)>0);
95 % %         if dr(ll,ii) ==0
96 % %             time(ll,ii) = -1;
```

```
97 % %           else
98 % %           aux = find(det(l1,:)==1);
99 % %           time(l1,ii) = MM(aux(1));
100 % %          end
101 % %
102 % %          end
103 %
104 % end
105 %
106
107
108
109
110
```

```
1 %%
2 load(['timeM_50db_M1' '.mat'], 'timeM')
3 load(['pareto_50db_M1' '.mat'], 'TXD', 'Mmax', 'parametros');
4 %%
5 load(['timeM_50db_M4' '.mat'], 'timeM')
6 load(['pareto_50db_M4' '.mat'], 'TXD', 'Mmax', 'parametros');
7
8 %%
9 % taxa de detecção x tempo
10 % timeM = time(binsM,:);
11 % timeM(timeM==-1) = Mmax;
12 % timeM = mean(timeM,1)'*1; %1segundo por janela
13 % TXD = mean(mean(Tdr(binsM, :, :),3),1)' *100;
14
15
16 % figure1 = figure;
17 axes1 = axes('Parent',figure1); hold(axes1,'on');
18 plot([0 1]*100, [Mmax Mmax], '-.k', 'linewidth',1)
19 plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k', 'linewidth',1)
20
21
22 for ii = 1:size(parametros,1)
23     plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',[num2str(
(parametros(ii,1)) '- ' num2str(parametros(ii,2))])
24 end
25
26 [ p, idxs] = paretoFront([TXD, (-timeM)] );
27 auxL = p(:,1)<0.5;
28 p(auxL,:) = [];
29 idxs(auxL,:) = [];
30
31 [~,ind] = sort(p(:,1));
32 p = p(ind,:);
33 idxs = idxs(ind,:);
34 % p([3,4,7,9],:) = [];
35 % idxs([3,4,7,9],:) = [];
36
37 plot(TXD(idxs),timeM(idxs),'-ok','Markersize',8,'linewidth',1.2)
38
39 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
40 % xlim([min(p(:,1))*80 max(p(:,1))*104])
41
42 set(axes1,'XMinorTick','on');
43 set(axes1,'YMinorTick','on');
44 box(axes1,'off');
45
46
47
```

```

48
49 for ii = 1:size(idxs,1)
50
51     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
52
53     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
54     fprintf(' NI = %d ',length(I));
55     I = I(1);
56
57     for jj = I
58
59         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
60         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
61         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);
62         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]);
63         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj,1))
',' num2str(parametros(jj,2)) '\}' ]);
64     end
65 end
66 fprintf('\n');
67 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
68 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
69
70 grid on
71 hold on
72
73 % %%
74 % hold on
75 % %%
76 % xlabel('Taxa de Detecção (%)','fontsize',12);
77 % ylabel('Tempo Médio de Exame (s)','fontsize',12);
78 % title(['Estímulo ', (Intensidade{1}), ': Método 04'])
79 % % title(['Estímulo ', (Intensidade{1}), ' SPL: Método 04'])
80
81

```

```
1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3
4 caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab'; 'An'; 'Bb'; 'Er'; 'Lu'; ...
8     'So'; 'Qu'; 'Vi'; 'Sa'; 'Ti'; 'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 %Intensidade -----
13 %intensidade = {'70dB'; '60dB'; '50dB'; '40dB'; '30dB'; 'ESP'}; %quais
intensidade analisadas
14 %sugestão
15 %vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'ESP'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 20; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 %parametros = [Min Mstep Mmax alfa_corrigido]
25 %parametros = [200 1 200 .05];
26 % nRuns = 10000;
27 % Mmax = 240; %número máximo de janela
28 % alfa_teste = 0.05;
29 % FP_desejado = 0.05;
30 % [alfa_corrigido, NDC_minimo, cost_alfa, P] =
funcao_NDC_alfaCorrigido_Mmax(nRuns, Mmax, alfa_teste, FP_desejado);
31 alfa = 0.05;
32 FP_desejado = 0.05;
33
34
35 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'], 'alfa_corrigido', ...
36     'NDC_minimo', 'P', 'nRuns')
37
38 parametros = [P, NDC_minimo, alfa_corrigido];
39 % parametros(:,5) = 0.05;
40
41
42 %%
43 load([caminho 'eletrodos.mat'])
```

```

44 pos_ele = 1;
45
46 ganho = 200;
47 alpha = 0.05;
48
49 %% -----
50 remoc = [.1]/ganho;
51
52
53 %%
54 %*****poder fazer por intensidade aqui -----
55
56 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
57
58     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o
voluntário
59     intensidade = cell2mat(Intensidade); %intensidadde
60     load([caminho voluntario intensidade],
'x','Fs','binsM','freqEstim')
61
62     x = x(:, :, pos_ele);
63
64     nfft = Fs;%1segundo de sinal
65
66     %retirar componente DC por janela (fiz isso pq no processamento em
67     %tempo real é por janela)
68     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada
trecho - devido a remoção
69
70     %scluir os dois primeiros segundos do inicio da coleta
71     x(:,1:2,:) = [];
72
73
74     %encontrar o valor máximo por canal
75     Vmax = max(abs(x), [], 1);
76     ind = Vmax>remoc;
77     [sum(ind) cont_vol ];
78     x = x(:,~ind); %remover o ruído amplitude
79     x = x(:,1:Mmax);%limitar o tamanho para o valor máximo.
80
81     %***** fazer por canal diferente ----for nCanal = 1:16 %
82     [dr,time] = protocolo_deteccao(x, parametros);
83
84     Tdr(:, :, cont_vol) = dr;
85     Ttime(:, :, cont_vol) = time;
86
87 end
88

```

```
89 %% Análise de desempenho
90
91 %TXD - analisar as freq. estimulação
92 %binsM = [82      84      86      88      90      92      94      96]
93 %freq. 81Hz,83,85,87,89,91,93,95Hz
94 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
95
96 binsR = binsM+1;
97 binsR = 1:100;
98 binsR(binsM) = [];
99 binsR(1:2) = [];
100
101 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
102
103
104 %% mostrar resultados
105 %clc
106 %1 - Taxa de detecção
107 figure
108 plot(TXD, '.k', 'MarkerSize', 10)
109 hold on
110 plot([0 size(TXD, 1)], [TXD(end) TXD(end)], ':r', 'LineWidth', 2)
111 ylabel('Taxa de Detecção', 'fontsize', 12)
112
113 %2 - Falsos Positivo
114 figure
115 plot(FP, '.k', 'MarkerSize', 10)
116 hold on
117 plot([0 size(FP, 1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2)
118 ylabel('Falso Positivo', 'fontsize', 12)
119
120 figure
121 boxplot(FP)
122
123 %%
124 % taxa de detecção x tempo
125 timeM = time(binsM, :);
126 timeM(timeM==-1) = Mmax;
127 timeM = mean(timeM, 1)'*1; %1segundo por janela
128 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)' *100;
129
130
131 figure1 = figure; axes1 = axes('Parent', figure1); hold(axes1, 'on');
132 plot([0 1]*100, [Mmax Mmax], '-.k', 'linewidth', 1)
133 plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k', 'linewidth', 1)
134
135
136 for ii = 1:size(parametros, 1)
```



```

137     plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',[num2str(
(parametros(ii,1)) '-' num2str(parametros(ii,2))])
138 end
139
140 [ p, idxs] = paretoFront([TXD,(-timeM)] );
141 auxL = p(:,1)<0.5;
142 p(auxL,:) = [];
143 idxs(auxL,:) = [];
144
145 [~,ind] = sort(p(:,1));
146 p = p(ind,:);
147 idxs = idxs(ind,:);
148 % p([3,4,7,9],:) = [];
149 % idxs([3,4,7,9],:) = [];
150
151 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
152
153 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
154 % xlim([min(p(:,1))*80 max(p(:,1))*104])
155
156 set(axes1,'XMinorTick','on');
157 set(axes1,'YMinorTick','on');
158 box(axes1,'off');
159 hold off
160 xlabel('Detection Rate (%)','fontsize',12);
161 ylabel('Mean Exam Time (min)','fontsize',12);
162 fprintf('\n');
163
164 for ii = 1:size(idxs,1)
165
166     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
167
168     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
169     fprintf(' NI = %d ',length(I));
170     I = I(1);
171
172     for jj = I
173
174         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros(
(jj,2));
175         %     text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros(
(jj,3)) '\}' ]]);
176 %         text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str(
(parametros(jj,3)) '\}' ]]);
177 %         text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]]);

```

```
178         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj),✓
1) ' ,' num2str(parametros(jj,2)) '\}' ]);
179     end
180 end
181 fprintf('\n');
182 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
183 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
184
185
186
187
```

```

1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3 %%
4 % caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab';'An';'Bb';'Er';'Lu';...
8     'So';'Qu';'Vi';'Sa';'Ti';'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 % Intensidade -----
13 % intensidade = {'70dB';'60dB';'50dB';'40dB';'30dB';'ESP'}; %quais
intensidade analisadas
14 % sugestão
15 % vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'50dB'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 240; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 %parametros = [Min Mstep Mmax alfa_corrigido]
25 %parametros = [200 1 200 .05];
26 % nRuns = 10000;
27 % Mmax = 240; %número máximo de janela
28 % alfa_teste = 0.05;
29 % FP_desejado =0.05;
30 % [alfa_corrigido,NDC_minimo,cost_alfa, P] =
funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax ,alfa_teste,FP_desejado);
31 alfa = 0.05;
32 FP_desejado = 0.05;
33
34
35 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
36     'NDC_minimo','P', 'nRuns')
37
38 parametros = [P, NDC_minimo,alfa_corrigido];
39
40
41 K_stages = zeros(size(parametros,1),1);
42 for ii = 1:numel(K_stages)
43     K_stages(ii) = numel(parametros(ii,1):parametros(ii,2):parametros

```

```

(ii,3));
44 end
45
46 parametros(K_stages<=5,:) = [];
47 %% Alfa constante
48 % deixar o alfa corrigido em 0.05 nesse caso não há correção
49 for ii=1:size(parametros,1)
50     parametros(ii,5) = 0.05;
51 end
52
53
54 %%
55 load(['eletrodos.mat'])
56 pos_ele = 1;
57
58 ganho = 200;
59 alpha = 0.05;
60
61 %% -----
62 remoc = [.1]/ganho;
63
64
65 %%
66 %*****poder fazer por intensidade aqui -----
67
68 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
69
70     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o ✓
voluntário
71     intensidade = cell2mat(Intensidade); %intensidadde
72     load([voluntario intensidade], 'x','Fs','binsM','freqEstim')
73
74     x = x(:, :, pos_ele);
75
76     nfft = Fs;%1segundo de sinal
77
78     %retirar componente DC por janela (fiz isso pq no processamento em
79     %tempo real é por janela)
80     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada ✓
trecho - devido a remoção
81
82     %scluir os dois primeiros segundos do inicio da coleta
83     x(:,1:2,:) =[];
84
85
86     %encontrar o valor máximo por canal
87     Vmax = max(abs(x), [], 1);
88     ind = Vmax>remoc;

```

```

89     [sum(ind) cont_vol ];
90     x = x(:,~ind); %remover o ruído amplitude
91     x = x(:,1:Mmax);%limitar o tamanho para o valor máximo.
92
93     %***** fazer por canal diferente ----for nCanal = 1:16 %
94     [dr,time] = protocolo_deteccao(x, parametros);
95
96     Tdr(:, :, cont_vol) = dr;
97     Ttime(:, :, cont_vol) = time;
98
99 end
100 % save(['Tdr_50db_M1' '.mat'], 'Tdr');
101 %% Análise de desempenho
102
103 %TXD - analisar as freq. estimulação
104 % binsM = [82      84      86      88      90      92      94      96]
105 %freq. 81Hz,83,85,87,89,91,93,95Hz
106 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
107 binsR = 70:104;
108 binsR(binsM - 71) = [];
109
110 % binsR = binsM+1;
111 % binsR = 1:100;
112 % binsR(1:2) = [];
113 % binsR(60) = [];
114
115 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
116
117 %% FP sob EEG ESP
118 % binsR= binsM;
119 %
120 % FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
121
122 %% mostrar resultados
123 %clc
124 %1 - Taxa de detecção
125
126 figure
127 plot(TXD*100, '.k', 'MarkerSize', 6) % antigo 10 [6]
128 hold on
129 % plot([0 size(TXD,1)], [TXD(end) TXD(end)], ':r', 'LineWidth', 2) %↙
single shot
130 ylabel('Taxa de Detecção', 'fontsize', 12)
131 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
132 ylabel('Taxa de Detecção (%)', 'fontsize', 12)
133 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
134 title(['Estímulo ', (Intensidade{1}), ' SPL: NNDC'])
135 % box off

```

```

136 grid on
137
138 %%
139 %2 - Falsos Positivo
140
141 figure
142 plot(FP*100, '.k', 'MarkerSize', 10)
143 hold on
144 % lsup = 7.41; 95%
145 % linf = 3.03; 95%
146 lsup = 6.73;
147 linf = 3.37;
148
149 plot([0 size(FP,1)], [lsup lsup], ':r', 'LineWidth', 2) %Limite do SUP✓
nível de significancia
150 % plot([0 size(FP,1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2) %✓
Limite do nível de significancia
151 plot([0 size(FP,1)], [linf linf], ':r', 'LineWidth', 2) %Limite do INF✓
nível de significancia
152 plot([0 size(FP,1)], [5 5], ':b', 'LineWidth', 2) %Limite do INF nível de✓
significancia
153
154
155 ylabel('Falso Positivo', 'fontsize', 12)
156 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
157 ylabel('Taxa de Falso Positivo (%)', 'fontsize', 12)
158 box off
159 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
160 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 01'])
161 %figure
162 % boxplot(FP)
163 grid on
164 %%
165
166 % taxa de detecção x tempo
167 timeM = time(binsM, :);
168 timeM(timeM == -1) = Mmax;
169 timeM = mean(timeM, 1) * 1; %1segundo por janela
170 TXD = mean(mean(Tdr(binsM, :, :), 3), 1) * 100;
171
172 %% Single Shot
173 figure1 = figure; axes1 = axes('Parent', figure1); hold(axes1, 'on');
174 plot([0 1]*100, [Mmax Mmax], '-.b', 'linewidth', 1)
175 plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
176
177
178 for ii = 1:size(parametros, 1)
179     plot(TXD(ii), timeM(ii), '.k', 'Markersize', 6, 'DisplayName', [num2str✓

```

```

(parametros(ii,1)) '-' num2str(parametros(ii,2))])
180 end
181
182 [ p, idxs] = paretoFront([TXD, (-timeM)] );
183 auxL = p(:,1)<0.5;
184 p(auxL,:) = [];
185 idxs(auxL,:) = [];
186
187 [~,ind] = sort(p(:,1));
188 p = p(ind,:);
189 idxs = idxs(ind,:);
190 % p([3,4,7,9],:) = [];
191 % idxs([3,4,7,9],:) = [];
192
193 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
194
195 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
196 % xlim([min(p(:,1))*80 max(p(:,1))*104])
197
198 set(axes1,'XMinorTick','on');
199 set(axes1,'YMinorTick','on');
200 box(axes1,'off');
201 hold off
202 xlabel('Taxa de Detecção (%)','fontsize',12);
203 ylabel('Tempo Médio de Exame (s)','fontsize',12);
204 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
205 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 01'])
206
207 fprintf('\n');
208
209 for ii = 1:size(idxs,1)
210
211     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
212
213     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
214     fprintf(' NI = %d ',length(I));
215     I = I(1);
216
217     for jj = I
218
219         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
220         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
221         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);

```

```
222 %         text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]);
223         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj,
1)) ',' num2str(parametros(jj,2)) '\}' ]);
224     end
225 end
226 fprintf('\n');
227 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
228 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
229
230 grid on
231 %% Salvar
232 % save(['timeM_30db_M1' '.mat'],'timeM')
233 % save(['pareto_30db_M1' '.mat'],'TXD','Mmax','parametros');
234
235 % save(['timeM_50db_M1' '.mat'],'timeM')
236 % save(['pareto_50db_M1' '.mat'],'TXD','Mmax','parametros');
237
238 % save(['Workspace_M1' '.mat']); %50dB
239 % save(['Workspace_M1_30' '.mat']); %30dB
240
```



```
1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3
4 % caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab';'An';'Bb';'Er';'Lu';...
8     'So';'Qu';'Vi';'Sa';'Ti';'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 %Intensidade -----
13 %intensidade = {'70dB';'60dB';'50dB';'40dB';'30dB';'ESP'}; %quais
intensidade analisadas
14 %sugestão
15 %vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'50dB'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 240; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 alfa = 0.05;
25 FP_desejado = 0.05;
26 % Dados gerados por Colatina montar matrix parametros ' [P, NDC_minimo,
alfa_corrigido]'
27
28 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
29     'NDC_minimo','P', 'nRuns')
30 parametros = [P, NDC_minimo,alfa_corrigido];
31 %% Alfa constante
32 % for ii=1:size(parametros,1)
33 %     parametros(ii,5) = 0.05;
34 % end
35
36
37 %%
38 load(['eletrodos.mat'])
39 pos_ele = 1; %
40
41 ganho = 200;
42 alpha = 0.05;
43
```

```

44 %% -----
45 remoc = [.1]/ganho;
46 %%
47 %*****poder fazer por intensidade aqui -----
48
49 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
50
51     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o ✓
voluntário
52     intensidade = cell2mat(Intensidade); %intensidadde
53     load([voluntario intensidade], 'x','Fs','binsM','freqEstim')
54
55     x = x(:, :, pos_ele);
56
57     nfft = Fs;%1segundo de sinal
58
59     %retirar componente DC por janela (fiz isso pq no processamento em
60     %tempo real é por janela)
61     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada ✓
trecho - devido a remoção
62
63     %scluir os dois primeiros segundos do inicio da coleta
64     x(:,1:2,:) =[];
65
66
67     %encontrar o valor máximo por canal
68     Vmax = max(abs(x), [], 1);
69     ind = Vmax>remoc;
70     [sum(ind) cont_vol ];
71     x = x(:, ~ind); %remover o ruído amplitude
72     x = x(:, 1:Mmax);%limitar o tamanho para o valor máximo.
73
74     %***** fazer por canal diferente ----for nCanal = 1:16 %
75     [dr,time] = protocolo_deteccao(x, parametros);
76
77     Tdr(:, :, cont_vol) = dr;
78     Ttime(:, :, cont_vol) = time;
79
80 end
81 %save(['Tdr_50db_M2' '.mat'], 'Tdr');
82 %% Análise de desempenho
83
84 %TXD - analisar as freq. estimulação
85 % binsM = [82      84      86      88      90      92      94      96 ];
86 % binsM2 = [binsM 2*binsM 3*binsM];
87
88 %%
89 %freq. 81Hz,83,85,87,89,91,93,95Hz

```

```
90 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
91 %%
92 % binsR = binsM+1;
93 % binsR = 1:100;
94 % binsR(binsM) = [];
95 % binsR(1:2) = [];
96
97
98 binsR = 70:104;
99 binsR(binsM - 71) = [];
100
101 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
102 %% FP sob EEG ESP
103 % binsR= binsM;
104 %
105 % FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
106
107 %% mostrar resultados
108 %clc
109 %1 - Taxa de detecção
110 figure
111 plot(TXD, '.k', 'MarkerSize', 6) % antigo 10 [6]
112 hold on
113 plot([0 size(TXD, 1)], [TXD(1) TXD(1)], ':r', 'LineWidth', 2) %single shot✓
first line
114 %ylabel('Taxa de Detecção', 'fontsize', 12)
115 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
116 ylabel('Taxa de Detecção (%)', 'fontsize', 12)
117 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
118 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
119 % box off
120 grid on
121
122 %%
123 %2 - Falsos Positivo
124 figure
125 plot(FP*100, '.k', 'MarkerSize', 10)
126 hold on
127 % lsup = 7.41; 95%
128 % linf = 3.03; 95%
129 lsup = 6.73;
130 linf = 3.37;
131
132 plot([0 size(FP, 1)], [lsup lsup ], ':r', 'LineWidth', 2) %Limite do SUP✓
nivel de significancia
133 % plot([0 size(FP, 1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2) %✓
Limite do nivel de significancia
134 plot([0 size(FP, 1)], [linf linf], ':r', 'LineWidth', 2) %Limite do INF✓
```

```

nivel de significancia
135
136 %ylabel('Falso Positivo','fontsize',12)
137 xlabel('Índice dos conjuntos de Parâmetros','fontsize',12)
138 ylabel('Taxa de Falso Positivo (%)','fontsize',12)
139 box off
140 % title([ (Intensidade{1}), 'ONTÂNEO: Método 02'])
141 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
142 %figure
143 ylim([0 inf])
144
145 % boxplot(FP)
146 grid on
147 %%
148 % taxa de detecção x tempo
149 timeM = time(binsM,:);
150 timeM(timeM==-1) = Mmax;
151 timeM = mean(timeM,1)*1; %1segundo por janela
152 TXD = mean(mean(Tdr(binsM, :, :), 3), 1) * 100;
153
154 %% Single Shot
155 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
156 plot([0 1]*100, [Mmax Mmax], '-.b', 'linewidth', 1)
157 plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
158 %Banco de dados com single shot na última linha
159 % plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
160
161
162 for ii = 1:size(parametros,1)
163     plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',[num2str(
parametros(ii,1)) '-' num2str(parametros(ii,2))])
164 end
165
166 [ p, idxs] = paretoFront([TXD, (-timeM)] );
167 auxL = p(:,1)<0.5;
168 p(auxL,:) = [];
169 idxs(auxL,:) = [];
170
171 [~,ind] = sort(p(:,1));
172 p = p(ind,:);
173 idxs = idxs(ind,:);
174 % p([3,4,7,9],:) = [];
175 % idxs([3,4,7,9],:) = [];
176
177 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
178
179 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])

```

```

180 % xlim([min(p(:,1))*80 max(p(:,1))*104])
181
182 set(axes1,'XMinorTick','on');
183 set(axes1,'YMinorTick','on');
184 box(axes1,'off');
185 hold off
186 xlabel('Taxa de Detecção (%)','fontsize',12);
187 ylabel('Tempo Médio de Exame (s)','fontsize',12);
188 % title([ (Intensidade{1}), 'ONTÂNEO: Método 02'])
189 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
190
191 fprintf('\n');
192
193 for ii = 1:size(idxs,1)
194
195     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
196
197     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
198     fprintf(' NI = %d ',length(I));
199     I = I(1);
200
201     for jj = I
202
203         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
204         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
205         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);
206         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]);
207         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj,
1)) ',' num2str(parametros(jj,2)) '\}' ]);
208     end
209 end
210 fprintf('\n');
211 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
212 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
213
214 grid on
215 save(['timeM_50db_M1' '.mat'],'timeM')
216 save(['pareto_50db_M1' '.mat'],'TXD','Mmax','parametros');
217 %% save(['Workspace_M2' '.mat']); %50dB
218 % save(['Workspace_M2_30' '.mat']); %30dB
219

```

```
1 function [NDC,FP] = estimarNDC(NDCinicial, alfa, FP_desejado, ord, Mmin, Mstep, Mmax)
2
3
4
5 %parametros de entrada
6 % NDCinicial = 10;
7 % Mmin = 10;
8 % Mstep = 1;
9 % Mmax = 30;
10 %ord ->vetor com as ORDs - cada linha uma simulação
11
12
13 MM = Mmin:Mstep:Mmax;
14 NNTmax = size(MM,2); %número de teste
15 nRuns = size(ord,1);
16 fprintf('Mmin =%d - Mstep=%d - Mmax=%d - Nruns=%d -NDCinicial=%d\n', Mmin, Mstep, Mmax, nRuns, NDCinicial);
17
18 %calcular o falso positivo
19 dr = zeros(nRuns,1);
20 FP =0;
21 for ii = 1:nRuns
22     dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDCinicial);
23 end
24 FP = mean(dr);
25
26 if (FP<FP_desejado)
27     if NDCinicial>3 %%mudei 1 --> 3
28         warning('NDC inicial grande --- Isso pode aumentar o tempo do código');
29         NDCinicial = 1;
30     end
31 end
32
33 FP = 0;
34 for NDC = NDCinicial:NNTmax %número máximo de teste
35
36     for ii = 1:nRuns
37         dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDC);
38     end
39     FP(NDC) = mean(dr);
40
41     if (FP(NDC)<FP_desejado)
42         break;
43     end
44
45 end
```

```
46
47
48 %encontrar o valor mínimo
49 [~,ind] = find((FP<FP_desejado).*(FP~=0)); %encontrar o primeiro NT que✓
%respeita a restrição
50
51 NDC = 0;
52 if isempty(ind)
53     NDC = nan;
54     warning('##### Não encontro Nalpha #####');
55     elseif ind(1)>1
56         NDC = interp1(FP([ind(1)-1,ind(1)]),[ind(1)-1,ind(1)],✓
FP_desejado);
57     else
58         NDC = 1; %para apenas uma detecção
59 end
60
61 if(NNTmax ==2)
62     NDC=2;
63 end
64
65 if(NNTmax ==1)
66     NDC=1;
67 end
68
69
```

```
1 function [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
2 % Minimize a continuous differentiable multivariate function. Starting
point
3 % is given by "X" (D by 1), and the function named in the string "f",
must
4 % return a function value and a vector of partial derivatives. The
Polack-
5 % Ribiere flavour of conjugate gradients is used to compute search
directions,
6 % and a line search using quadratic and cubic polynomial approximations
and the
7 % Wolfe-Powell stopping criteria is used together with the slope ratio
method
8 % for guessing initial step sizes. Additionally a bunch of checks are
made to
9 % make sure that exploration is taking place and that extrapolation
will not
10 % be unboundedly large. The "length" gives the length of the run: if it
is
11 % positive, it gives the maximum number of line searches, if negative
its
12 % absolute gives the maximum allowed number of function evaluations.
You can
13 % (optionally) give "length" a second component, which will indicate
the
14 % reduction in function value to be expected in the first line-search
(defaults
15 % to 1.0). The function returns when either its length is up, or if no
further
16 % progress can be made (ie, we are at a minimum, or so close that due
to
17 % numerical problems, we cannot get any closer). If the function
terminates
18 % within a few iterations, it could be an indication that the function
value
19 % and derivatives are not consistent (ie, there may be a bug in the
20 % implementation of your "f" function). The function returns the found
21 % solution "X", a vector of function values "fX" indicating the
progress made
22 % and "i" the number of iterations (line searches or function
evaluations,
23 % depending on the sign of "length") used.
24 %
25 % Usage: [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
26 %
27 % See also: checkgrad
28 %
29 % Copyright (C) 2001 and 2002 by Carl Edward Rasmussen. Date 2002-02-13
```



```
30 %
31 %
32 % (C) Copyright 1999, 2000 & 2001, Carl Edward Rasmussen
33 %
34 % Permission is granted for anyone to copy, use, or modify these
35 % programs and accompanying documents for purposes of research or
36 % education, provided this copyright notice is retained, and note is
37 % made of any changes that have been made.
38 %
39 % These programs and documents are distributed without any warranty,
40 % express or implied. As the programs were written for research
41 % purposes only, they have not been tested to the degree that would be
42 % advisable in any important application. All use of these programs is
43 % entirely at the user's own risk.
44 %
45 % [ml-class] Changes Made:
46 % 1) Function name and argument specifications
47 % 2) Output display
48 %
49
50 % Read options
51 if exist('options', 'var') && ~isempty(options) && isfield(options, 'MaxIter', ✓
'MaxIter')
52     length = options.MaxIter;
53 else
54     length = 100;
55 end
56
57
58 RHO = 0.01; % a bunch of constants for line ✓
searches
59 SIG = 0.5; % RHO and SIG are the constants in the Wolfe-Powell ✓
conditions
60 INT = 0.1; % don't reevaluate within 0.1 of the limit of the current ✓
bracket
61 EXT = 3.0; % extrapolate maximum 3 times the current ✓
bracket
62 MAX = 20; % max 20 function evaluations per ✓
line search
63 RATIO = 100; % maximum allowed ✓
slope ratio
64
65 argstr = ['feval(f, X']; % compose string used to ✓
call function
66 for i = 1:(nargin - 3)
67     argstr = [argstr, ',P', int2str(i)];
68 end
69 argstr = [argstr, ')]';
```

```

70
71 if max(size(length)) == 2, red=length(2); length=length(1); else red=1;✓
end
72 S=['Iteration '];
73
74 i = 0; % zero the run length✓
counter
75 ls_failed = 0; % no previous line search✓
has failed
76 fX = [];
77 [f1 df1] = eval(argstr); % get function value and✓
gradient
78 i = i + (length<0); % count✓
epochs?!
79 s = -df1; % search direction is✓
steepest
80 d1 = -s'*s; % this is✓
the slope
81 z1 = red/(1-d1); % initial step is✓
red/(|s|+1)
82
83 while i < abs(length) % while not✓
finished
84 i = i + (length>0); % count✓
iterations?!
85
86 X0 = X; f0 = f1; df0 = df1; % make a copy of✓
current values
87 X = X + z1*s; % begin✓
line search
88 [f2 df2] = eval(argstr);
89 i = i + (length<0); % count✓
epochs?!
90 d2 = df2'*s;
91 f3 = f1; d3 = d1; z3 = -z1; % initialize point 3 equal to✓
point 1
92 if length>0, M = MAX; else M = min(MAX, -length-i); end
93 success = 0; limit = -1; % initialize quantities
94 while 1
95 while ((f2 > f1+z1*RHO*d1) || (d2 > -SIG*d1)) && (M > 0)
96 limit = z1; % tighten the✓
bracket
97 if f2 > f1
98 z2 = z3 - (0.5*d3*z3*z3)/(d3*z3+f2-f3); %✓
quadratic fit
99 else
100 A = 6*(f2-f3)/z3+3*(d2+d3); %✓
cubic fit

```

```

101         B = 3*(f3-f2)-z3*(d3+2*d2);
102         z2 = (sqrt(B*B-A*d2*z3*z3)-B)/A;           % numerical error✓
possible - ok!
103         end
104         if isnan(z2) || isinf(z2)
105             z2 = z3/2;                             % if we had a numerical problem✓
then bisect
106         end
107         z2 = max(min(z2, INT*z3), (1-INT)*z3); % don't accept too close✓
to limits
108         z1 = z1 + z2;                               % update✓
the step
109         X = X + z2*s;
110         [f2 df2] = eval(argstr);
111         M = M - 1; i = i + (length<0);               % count✓
epochs?!
112         d2 = df2'*s;
113         z3 = z3-z2;                                % z3 is now relative to the✓
location of z2
114         end
115         if f2 > f1+z1*RHO*d1 || d2 > -SIG*d1
116             break;                                   % this is a✓
failure
117         elseif d2 > SIG*d1
118             success = 1; break;                       %✓
success
119         elseif M == 0
120             break;                                   %✓
failure
121         end
122         A = 6*(f2-f3)/z3+3*(d2+d3);                 % make cubic✓
extrapolation
123         B = 3*(f3-f2)-z3*(d3+2*d2);
124         z2 = -d2*z3*z3/(B+sqrt(B*B-A*d2*z3*z3));    % num. error✓
possible - ok!
125         if ~isreal(z2) || isnan(z2) || isinf(z2) || z2 < 0 % num prob or✓
wrong sign?
126             if limit < -0.5                         % if we have no✓
upper limit
127                 z2 = z1 * (EXT-1);                 % the extrapolate the✓
maximum amount
128             else
129                 z2 = (limit-z1)/2;                   %✓
otherwise bisect
130             end
131             elseif (limit > -0.5) && (z2+z1 > limit) % extrapolation✓
beyond max?
132             z2 = (limit-z1)/2;✓

```

[illegible]

```
give up
164     end
165     tmp = df1; df1 = df2; df2 = tmp;                % swap✓
derivatives
166     s = -df1;                                       % try✓
steepest
167     d1 = -s'*s;
168     z1 = 1/(1-d1);
169     ls_failed = 1;                                % this line✓
search failed
170     end
171     if exist('OCTAVE_VERSION')
172         fflush(stdout);
173     end
174 end
175 fprintf('\n');
176
```

```
1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido, cost_alfa, P] =funcao_alfaCorrigido_Mmax(nRuns, ✓
Mmax, FP_desejado)
6 %parametros de entrada -----
7 % pentrada = ;
8 FP_desejado = 0.05;
9 nRuns = 10;
10 Mmax = 440; %número máximo de janela
11 %-----
12 %parâmetros default
13 % fs = 64;
14 tj = 32; %cada janela um segundo
15 bin = 8;
16
17 Ntotal = Mmax*tj; %número de pontos totais
18
19 %Na simulação iremos estimar a aplicação do detector a cada janela
20 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
21
22 for ii = 1: nRuns
23     x = randn(Ntotal,1);
24     x = reshape(x,tj,Mmax); %dividir em janelas
25     %aplicar o detector a cada janela -----
26     xfft = fft(x); %aplico uma única vez a FFT.
27     for M = 2:Mmax %fazer para cada acrescimo de uma janela
28         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
29     end
30 end
31
32
33 %% obter todos os parâmetros
34 P = parametros_protocolo(Mmax);
35 % P=pentrada;
36 alfa_corrigido = nan*ones(size(P,1),1);
37 cost_alfa = nan*ones(size(P,1),1);
38
39 for ii = 1:size(P,1)
40     Mmin = P(ii,1);
41     Mstep = P(ii,2);
42     Mmax = P(ii,3);
43     MM = Mmin:Mstep:Mmax;
44     disp([num2str(ii*100/size(P,1)), '%'])
45
46     det = ord(:,MM);
```

```
47     alfa = 0.05; %TAXA DE FALSO POSITIVO DE CADA TESTES
48     options = optimset('MaxIter', 50);
49     cc = @(alfa) funcao_custo(alfa ,MM, det, FP_desejado);
50     [alfa, cost] = fmincg(cc,alfa, options);
51     alfa_corrigido(ii) = alfa;
52
53     if ~isempty(cost)
54         cost_alfa(ii) = cost(end);
55     end
56
57 end
58 %salvar as variáveis
59 save(['NDC_Unitario_Mmax_' num2str(Mmax) '_alfa_' num2str(alfa) ✓
'_FPdesejado' num2str(FP_desejado) '.mat'], 'alfa_corrigido', ...
60     'NDC_minimo', 'P', 'nRuns', 'alfa', 'FP_desejado')
61
62 t1 = table(P, NDC_minimo, alfa_corrigido, cost_alfa);
63 % figure
64 % plot(alfa_corrigido, 'ok')
65 % figure
66 % plot(cost_alfa, 'ok')
67 %ylim([min(cost_alfa) max(cost_alfa)])
68
69
70
71
72
73
```

```

1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido, cost_alfa, P] =funcao_alfaCorrigido_Mmax(nRuns, ✓
Mmax, FP_desejado)
6 %parametros de entrada -----
7 % pentrada = ;
8 FP_desejado = 0.05;
9 nRuns = 100;
10 Mmax = 20; %número máximo de janela
11 alfa_teste = 0.05;
12 %-----
13 %parâmetros default
14 % fs = 64;
15 tj = 32; %cada janela um segundo
16 bin = 8;
17
18 Ntotal = Mmax*tj; %número de pontos totais
19
20 %Na simulação iremos estimar a aplicação do detector a cada janela
21 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
22
23 for ii = 1: nRuns
24     x = randn(Ntotal,1);
25     x = reshape(x,tj,Mmax); %dividir em janelas
26     %aplicar o detector a cada janela -----
27     xfft = fft(x); %aplico uma única vez a FFT.
28     for M = 2:Mmax %fazer para cada acrescimo de uma janela
29         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
30     end
31 end
32
33
34 %% obter todos os parâmetros
35 P = parametros_protocolo(Mmax);
36 % P=pentrada;
37 alfa_corrigido = nan*ones(size(P,1),1);
38 cost_alfa = nan*ones(size(P,1),1);
39
40 for ii = 1:size(P,1)
41     Mmin = P(ii,1);
42     Mstep = P(ii,2);
43     Mmax = P(ii,3);
44     MM = Mmin:Mstep:Mmax;
45     disp([num2str(ii*100/size(P,1)), '%'])
46

```



```
47     det = ord(:,MM);
48     alfa = 0.05; %TAXA DE FALSO POSITIVO DE CADA TESTES
49     options = optimset('MaxIter', 50);
50     cc = @(alfa) funcao_custo(alfa ,MM, det, FP_desejado);
51     [alfa, cost] = fmincg(cc,alfa, options);
52     alfa_corrigido(ii) = alfa;
53
54     if ~isempty(cost)
55         cost_alfa(ii) = cost(end);
56     end
57
58 end
59
60 %salvar as variáveis
61 alfa = alfa_teste; % eu coloquei
62 save(['NDC_Unitario_Mmax_' num2str(Mmax) '_alfa_' num2str(alfa) ✓
63     '_FPdesejado' num2str(FP_desejado) '.mat'], 'alfa_corrigido', ...
64     'P', 'nRuns', 'alfa', 'FP_desejado')
65
66 t1 = table(P, alfa_corrigido, cost_alfa);
67 % figure
68 % plot(alfa_corrigido, 'ok')
69 % figure
70 % plot(cost_alfa, 'ok')
71 %ylim([min(cost_alfa) max(cost_alfa)])
72
73
74
75
76
```

```
1 function [J,grad] = funcao_custo(alfa,MM,det, FP_desejado)
2
3 nRuns = size(det,1);
4 %1-Critério de parada -----
5 valor_critico = VC_MSC(MM,alfa);
6 det = det>repmat(valor_critico,nRuns,1);
7 FP = mean((sum(det,2)>0)); %TAXA DE FALSO POSITIVO DA SEÇÃO (EXAME)
8
9 %2 - Cálculo da função de custo -----
10 erro = (FP-FP_desejado);
11 %J = erro.^2;
12 J = erro.^2+100*(alfa<0);
13
14
15 %3 - Cálculo do gradiente -----
16 %grad = erro*FP;
17 grad = erro*FP; %~porporcional ao gradiente (não é o verdadeiro - mas✓
porporcional)
18
19
```

```
1 function [J,grad] = funcao_custo_v2(alfa, NDC, MM, ord, FP_desejado)
2
3 nRuns = size(ord,1);
4 alfa = max(alfa,eps);
5 alfa = min(alfa,1-eps);
6 %1-Critério de parada -----
7 dr = zeros(nRuns,1);
8 for ii = 1:nRuns
9     dr(ii,1) = ETS(ord(ii,:),MM,alfa,NDC);
10 end
11 FP = mean(dr);
12
13 %2 - Cálculo da função de custo -----
14 erro = (FP-FP_desejado);
15 %J = erro.^2;
16 J = 1/2*erro.^2;
17
18
19 %3 - Cálculo do gradiente -----
20 grad = erro;
21
22 %grad = erro*FP; %~porporcional ao gradiente (não é o verdadeiro - mas ✓
porporcional)
23
```

```

1 %função obter o alfa para todos os Mmax -----
2 %-----
3 %%function funcao_alfaCorrigido_Mmax
4 % Aplicação do protocolo de detecção na condição de Ho -----
5 function [alfa_corrigido,NDC_minimo,cost_alfa, P] ✓
=funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax,alfa_teste,FP_desejado)
6 %parametros de entrada -----
7 % clear all, close all, clc
8 nRuns = 1000;
9 Mmax = 20; %número máximo de janela
10 alfa_teste = 0.05;
11 FP_desejado =0.05;
12 %-----
13
14 %parâmetros default
15 % fs = 64;
16 tj = 32; %cada janela um segundo
17 bin = 8;
18
19
20 Ntotal = Mmax*tj; %número de pontos totais
21
22 %Na simulação iremos estimar a aplicação do detector a cada janela
23 ord = zeros(nRuns,Mmax); %armazena os valores dos detectores a cada ✓
experimento.
24
25 for ii = 1: nRuns
26     x = randn(Ntotal,1);
27     x = reshape(x,tj,Mmax); %dividir em janelas
28     %aplicar o detector a cada janela -----
29     xfft = fft(x); %aplico uma única vez a FFT.
30     for M = 2:Mmax %fazer para cada acrescimo de uma janela
31         ord(ii,M) = msc_fft(xfft(bin,1:M),M);
32     end
33 end
34
35
36 %% obter todos os parâmetros
37 P = parametros_protocolo(Mmax);
38 alfa_corrigido = nan*ones(size(P,1),1);
39 cost_alfa = nan*ones(size(P,1),1);
40
41 NDC_minimo = 3+nan*ones(size(P,1),1); %NDC_mínimo
42
43 %%
44 [~,aux] = sort(ceil((Mmax - P(:,1))./P(:,2))+1);
45 P = P(aux,:);
46

```

```

47
48 for ii = 1:size(P,1)
49     Mmin = P(ii,1);
50     Mstep = P(ii,2);
51     Mmax = P(ii,3);
52     MM = Mmin:Mstep:Mmax;
53     disp([num2str(ii*100/size(P,1)),'%'])
54
55 %         %      NDC:sugerir um bom
56 if ii == 1
57     Ninicial = 1;
58 else
59     Ninicial = max(round(NDC_minimo(ii-1))-5,1); %% Mudei 3--> 5
60 end
61
62
63 [NDC,FP] = estimarNDC(Ninicial,alfa_teste,FP_desejado, ord, Mmin,✓
Mstep, Mmax);
64 NDC_minimo(ii) = NDC;
65 %     NDC_minimo(ii) = 1;
66
67 %ajustar os valores crítico
68 MM = Mmin:Mstep:Mmax;
69 options = optimset('MaxIter', 50);
70 cc = @(alfa) funcao_custo_v2(alfa, NDC_minimo(ii), MM, ord,✓
FP_desejado);
71 [alfa, cost] = fmincg(cc,alfa_teste, options);
72 alfa_corrigido(ii) = alfa;
73
74 %     if isempty(cost)
75 %         y= 1;
76 %     end
77
78 if ~isempty(cost)
79     cost_alfa(ii) = cost(end);
80 end
81
82
83 end
84
85 % alfa = alfa_teste;
86
87 % save('NNDC_AlfaCorrigido_Mmax20_alfa_0.05_FPdesejado0.05.✓
mat','alfa','alfa_corrigido','FP_desejado', 'NDC_minimo', 'P', 'nRuns');
88 % save(['NNDC_AlfaCorrigido_Mmax_' num2str(Mmax) '_alfa_' num2str✓
(alfa_teste) '_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido',✓
...
89 %     'P', 'nRuns','alfa','FP_desejado')

```

```
90 % figure
91 % plot(alfa_corrigido,'ok')
92 % figure
93 % plot(cost_alfa,'ok')
94 % ylim([min(cost_alfa) max(cost_alfa)])
95
96
97
98
99
100
```

```
1
2 %%
3 clearvars
4 linha = [line(),line(),line(),line()];
5 close all
6 clc
7
8 %%
9 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
10
11 cor = ['m','r','g','b'];
12
13 for metodo = 1:4
14     caminho1 = ['timeM_30db_M',num2str(metodo),'.mat'];
15     caminho2 = ['pareto_30db_M',num2str(metodo),'.mat'];
16
17     load(caminho1,'timeM')
18     load(caminho2,'TXD','Mmax','parametros');
19
20     % pontos e linhas pretos:
21     plot([0 1]*100, [Mmax Mmax], '-.k','linewidth',1)
22     plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k','linewidth',1)
23
24
25     for ii = 1:size(parametros,1)
26         plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',
27             [num2str(parametros(ii,1)) '-' num2str(parametros(ii,2))])
28     end
29
30     [p, idxs] = paretoFront([TXD,(-timeM)]);
31     auxL = p(:,1)<0.5;
32     p(auxL,:) = [];
33     idxs(auxL,:) = [];
34
35     [~,ind] = sort(p(:,1));
36     p = p(ind,:);
37     idxs = idxs(ind,:);
38
39     % Linha colorida:
40     tipo = ['-o' cor(metodo)];
41     linha(metodo) = plot(TXD(idxs),timeM(idxs),tipo,'Markersize',
42         8,'linewidth',1.8);
43
44     for ii = 1:size(idxs,1)
45         [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
46         I = I(1);
```

```
46
47     for jj = I
48         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(
49 (jj,1)) ',' num2str(parametros(jj,2)) '\}' ]);
50     end
51 end
52
53 %%
54 set(axes1,'XMinorTick','on');
55 set(axes1,'YMinorTick','on');
56 box(axes1,'off');
57
58
59 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
60 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
61
62 grid on
63 hold on
64 title('Curva de Pareto - INTENSIDADE 30 dB SPL')
65 legend([linha(1),linha(2),linha(3),linha(4)],'Método 1','Método 2',
66 'Método 3','Método 4')
67 xlabel('Taxa de detecção [%]')
68 ylabel('Tempo de exame [s]')
```



```
1 function P = parametros_protocolo(Mmax)
2
3 %função para obter todos os parâmetros possíveis de um protocolo de
4 %detecção a partir de um Mmax de entrada
5
6 %entrada: Mmax
7
8 %saída: Matriz P onde:
9         %cada linha indica um parâmetro.
10        %Coluna [ Min, Mstep, Mmax];
11
12 %%Código -----
13
14 P = [];
15
16 for M_step = 1:(Mmax-1)
17     for Mmin = 2:(Mmax-1)%MMentrada
18         k = (Mmax -Mmin)/M_step; %NTmax:nº max de teste ORD no✓
19         if (ceil(k) == floor(k) & k>=0) %verificar se o valor de k é um✓
20             P = [P;Mmin,M_step,Mmax];
21         end
22     end
23 end
24
25 %Incluir o teste único também
26 P = [P;Mmax 1 Mmax];
27
28
29
30
31
```

```
1 %%
2 clearvars
3 linha = [line(),line(),line(),line()];
4 close all
5 clc
6
7 %%
8 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
9 Intensidade = {'30dB'};
10 % Intensidade = {'50dB'};
11 cor = {'m','r','g','b'};
12
13 for metodo = 1:4
14     caminho1 = ['timeM_50db_M',num2str(metodo),'.mat'];
15     caminho2 = ['pareto_50db_M',num2str(metodo),'.mat'];
16 %     caminho1 = ['timeM_30db_M',num2str(metodo),'.mat'];
17 %     caminho2 = ['pareto_30db_M',num2str(metodo),'.mat'];
18
19     load(caminho1,'timeM')
20     load(caminho2,'TXD','Mmax','parametros');
21
22     % pontos e linhas pretos: SINGLE SHOT 50dB
23     single = plot([0 1]*100, [Mmax Mmax], '-.b','linewidth',1);
24 %     plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b','linewidth',1)
25 % SINGLE SHOT 30dB
26     plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.b','linewidth',1);
27
28
29
30     for ii = 1:size(parametros,1)
31         plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',
[num2str(parametros(ii,1)) '-' num2str(parametros(ii,2))])
32     end
33
34     [p, idxs] = paretoFront([TXD,(-timeM)]);
35     auxL = p(:,1)<0.5;
36     p(auxL,:) = [];
37     idxs(auxL,:) = [];
38
39     [~,ind] = sort(p(:,1));
40     p = p(ind,:);
41     idxs = idxs(ind,:);
42
43     % Linha colorida:
44     tipo = ['-o' cor(metodo)];
45     linha(metodo) = plot(TXD(idxs),timeM(idxs),tipo,'Markersize',
8,'linewidth',1.8);
```

```
46
47     for ii = 1:size(idxs,1)
48
49         [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
50         I = I(1);
51
52         for jj = I
53             text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(
54 (jj,1)) '\,' num2str(parametros(jj,2)) '\}' ]);
55         end
56     end
57
58 %%
59 set(axes1,'XMinorTick','on');
60 set(axes1,'YMinorTick','on');
61 box(axes1,'off');
62
63 %
64 % xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
65 % ylim([min(timeM(idxs))*0.95 Mmax*1.05])
66
67 grid on
68 hold on
69 % title(['Curva de Pareto Intensidade ', (Intensidade{1}), ' SPL: Métodos 01 a 04'])
70 title(['Curva de Pareto Intensidade ', (Intensidade{1}), ' SPL: Métodos NND'])
71 % title('Curva de Pareto - 30 dB')
72
73 % legend([linha(1),linha(2),linha(3),linha(4), single],'Método 1','Método 2','Método 3','Método 4','Single test','Location','southeast')
74 legend([linha(1),linha(2),linha(3),linha(4), single],'NND = 1','NND = 2','NND = 3','NND = 5','Single test','Location','southeast')
75
76 xlabel('Taxa de detecção [%]')
77 ylabel('Tempo de exame [s]')
78
```

```
1 function [ p, idxs] = paretoFront( p )
2 % Filters a set of points P according to Pareto dominance, i.e., points
3 % that are dominated (both weakly and strongly) are filtered.
4 %
5 % Inputs:
6 % - P      : N-by-D matrix, where N is the number of points and D is the
7 %            number of elements (objectives) of each point.
8 %
9 % Outputs:
10 % - P      : Pareto-filtered P
11 % - idxs   : indices of the non-dominated solutions
12 %
13 % Example:
14 % p = [1 1 1; 2 0 1; 2 -1 1; 1, 1, 0];
15 % [f, idxs] = paretoFront(p)
16 %      f = [1 1 1; 2 0 1]
17 %      idxs = [1; 2]
18
19 [i, dim] = size(p);
20 idxs = [1 : i]';
21 while i >= 1
22     old_size = size(p,1);
23     indices = sum( bsxfun( @ge, p(i,:), p ), 2 ) == dim;
24     indices(i) = false;
25     p(indices,:) = [];
26     idxs(indices) = [];
27     i = i - 1 - (old_size - size(p,1)) + sum(indices(i:end));
28 end
29
30 end
```

```
1 function [dr,time] = protocolo_deteccao(x, parametros)
2 % mmaxx = 200*ones(size (parametros));
3 binsM = 120;
4 %c  cular o valor do detector
5 %aplicar o detector a cada janela -----
6 xfft = fft(x); %aplico uma   nica vez a FFT.
7
8 for M = 2:size(x,2) %fazer para cada acrescimo de uma janela
9     % ord(:,M) = msc_fft(xfft(bin,1:M),M);
10    ord(:,M) = msc_fft(xfft(1:binsM,1:M),M);
11 end
12
13
14 for ii = 1:size(parametros,1)
15
16     %parametros
17     Mmin = parametros(ii,1);
18     Mstep = parametros(ii,2);
19     Mmax = parametros(ii,3);
20     NDC = parametros(ii,4);
21     alfa = parametros(ii,5);
22
23     % protocolo
24     MM = Mmin:Mstep:Mmax;
25
26     if M <240
27         disp('aha')
28     end
29
30     for ll = 1:size(ord,1)
31
32         [dr(ll,ii),time(ll,ii)] = ETS(ord(ll,:),MM',alfa,NDC);
33     end
34
35 %     det = ord(:,MM);
36 %
37 %     valor_critico = VC_MSC(MM,alfa);
38 %     det = det>repmat(valor_critico,size(det,1),1);
39 %
40 %     %dr = double(sum(det,2)>1);
41 %     for ll = 1:size(ord,1)
42 %
43 %         dr(ll,ii) = double(sum(det(ll,:),2)>0);
44 %         if dr(ll,ii) ==0
45 %             time(ll,ii) = -1;
46 %         else
47 %             aux = find(det(ll,')==1);
48 %             time(ll,ii) = MM(aux(1));
```

```
49 %         end
50 %
51 %     end
52
53 end
54
55 % %% Não detecção
56 % function [dr,time] = protocolo_deteccao(x, parametros)
57 %
58 % binsM = 120;
59 % %cálculo o valor do detector
60 % %aplicar o detector a cada janela -----
61 % xfft = fft(x); %aplico uma única vez a FFT.
62 %
63 % for M = 2:size(x,2) %fazer para cada acrescimo de uma janela
64 %     % ord(:,M) = msc_fft(xfft(bin,1:M),M);
65 %     ord(:,M) = msc_fft(xfft(1:binsM,1:M),M);
66 % end
67 %
68 %
69 % for ii = 1:size(parametros,1)
70 %
71 %     %parametros
72 %     Mmin = parametros(ii,1);
73 %     Mstep = parametros(ii,2);
74 %     Mmax = parametros(ii,3);
75 %     NDC = parametros(ii,4);
76 %     alfa = parametros(ii,5);
77 %
78 %     % protocolo
79 %     MM = Mmin:Mstep:Mmax;
80 %
81 %     for ll = 1:size(ord,1)
82 %
83 %         [dr(ll,ii),time(ll,ii)] = ETS(ord(ll,:),MM',alfa,NDC);
84 %     end
85 %
86 % %     det = ord(:,MM);
87 % %
88 % %     valor_critico = VC_MSC(MM,alfa);
89 % %     det = det>repmat(valor_critico,size(det,1),1);
90 % %
91 % % %     %dr = double(sum(det,2)>1);
92 % %     for ll = 1:size(ord,1)
93 % %
94 % %         dr(ll,ii) = double(sum(det(ll,:),2)>0);
95 % %         if dr(ll,ii) ==0
96 % %             time(ll,ii) = -1;
```

```
97 % %           else
98 % %           aux = find(det(l1,:)==1);
99 % %           time(l1,ii) = MM(aux(1));
100 % %          end
101 % %
102 % %          end
103 %
104 % end
105 %
106
107
108
109
110
```

```
1 %%
2 load(['timeM_50db_M1' '.mat'], 'timeM')
3 load(['pareto_50db_M1' '.mat'], 'TXD', 'Mmax', 'parametros');
4 %%
5 load(['timeM_50db_M4' '.mat'], 'timeM')
6 load(['pareto_50db_M4' '.mat'], 'TXD', 'Mmax', 'parametros');
7
8 %%
9 % taxa de detecção x tempo
10 % timeM = time(binsM,:);
11 % timeM(timeM==-1) = Mmax;
12 % timeM = mean(timeM,1)'*1; %1segundo por janela
13 % TXD = mean(mean(Tdr(binsM, :, :), 3), 1)' *100;
14
15
16 % figure1 = figure;
17 axes1 = axes('Parent', figure1); hold(axes1, 'on');
18 plot([0 1]*100, [Mmax Mmax], '-.k', 'linewidth', 1)
19 plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k', 'linewidth', 1)
20
21
22 for ii = 1:size(parametros,1)
23     plot(TXD(ii), timeM(ii), '.k', 'Markersize', 6, 'DisplayName', [num2str(
(parametros(ii,1)) '-' num2str(parametros(ii,2))])
24 end
25
26 [ p, idxs] = paretoFront([TXD, (-timeM)] );
27 auxL = p(:,1)<0.5;
28 p(auxL,:) = [];
29 idxs(auxL,:) = [];
30
31 [~,ind] = sort(p(:,1));
32 p = p(ind,:);
33 idxs = idxs(ind,:);
34 % p([3,4,7,9],:) = [];
35 % idxs([3,4,7,9],:) = [];
36
37 plot(TXD(idxs), timeM(idxs), '-ok', 'Markersize', 8, 'linewidth', 1.2)
38
39 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
40 % xlim([min(p(:,1))*80 max(p(:,1))*104])
41
42 set(axes1, 'XMinorTick', 'on');
43 set(axes1, 'YMinorTick', 'on');
44 box(axes1, 'off');
45
46
47
```



```

48
49 for ii = 1:size(idxs,1)
50
51     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
52
53     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
54     fprintf(' NI = %d ',length(I));
55     I = I(1);
56
57     for jj = I
58
59         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
60         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
61         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);
62         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]);
63         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj,1))
',' num2str(parametros(jj,2)) '\}' ]);
64     end
65 end
66 fprintf('\n');
67 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
68 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
69
70 grid on
71 hold on
72
73 % %%
74 % hold on
75 % %%
76 % xlabel('Taxa de Detecção (%)','fontsize',12);
77 % ylabel('Tempo Médio de Exame (s)','fontsize',12);
78 % title(['Estímulo ', (Intensidade{1}), ': Método 04'])
79 % % title(['Estímulo ', (Intensidade{1}), ' SPL: Método 04'])
80
81

```

```

1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3
4 caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab';'An';'Bb';'Er';'Lu';...
8     'So';'Qu';'Vi';'Sa';'Ti';'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 %Intensidade -----
13 %intensidade = {'70dB';'60dB';'50dB';'40dB';'30dB';'ESP'}; %quais
intensidade analisadas
14 %sugestão
15 %vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'ESP'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 20; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 %parametros = [Min Mstep Mmax alfa_corrigido]
25 %parametros = [200 1 200 .05];
26 % nRuns = 10000;
27 % Mmax = 240; %número máximo de janela
28 % alfa_teste = 0.05;
29 % FP_desejado =0.05;
30 % [alfa_corrigido,NDC_minimo,cost_alfa, P] =
funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax ,alfa_teste,FP_desejado);
31 alfa = 0.05;
32 FP_desejado = 0.05;
33
34
35 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
36     'NDC_minimo','P', 'nRuns')
37
38 parametros = [P, NDC_minimo,alfa_corrigido];
39 % parametros(:,5) = 0.05;
40
41
42 %%
43 load([caminho 'eletrodos.mat'])

```

```

44 pos_ele = 1;
45
46 ganho = 200;
47 alpha = 0.05;
48
49 %% -----
50 remoc = [.1]/ganho;
51
52
53 %%
54 %*****poder fazer por intensidade aqui -----
55
56 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
57
58     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o
voluntário
59     intensidade = cell2mat(Intensidade); %intensidadde
60     load([caminho voluntario intensidade],
'x','Fs','binsM','freqEstim')
61
62     x = x(:, :, pos_ele);
63
64     nfft = Fs;%1segundo de sinal
65
66     %retirar componente DC por janela (fiz isso pq no processamento em
67     %tempo real é por janela)
68     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada
trecho - devido a remoção
69
70     %scluir os dois primeiros segundos do inicio da coleta
71     x(:,1:2,:) = [];
72
73
74     %encontrar o valor máximo por canal
75     Vmax = max(abs(x), [], 1);
76     ind = Vmax>remoc;
77     [sum(ind) cont_vol ];
78     x = x(:,~ind); %remover o ruído amplitude
79     x = x(:,1:Mmax);%limitar o tamanho para o valor máximo.
80
81     %***** fazer por canal diferente ----for nCanal = 1:16 %
82     [dr,time] = protocolo_deteccao(x, parametros);
83
84     Tdr(:, :, cont_vol) = dr;
85     Ttime(:, :, cont_vol) = time;
86
87 end
88

```

```
89 %% Análise de desempenho
90
91 %TXD - analisar as freq. estimulação
92 %binsM = [82      84      86      88      90      92      94      96]
93 %freq. 81Hz,83,85,87,89,91,93,95Hz
94 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
95
96 binsR = binsM+1;
97 binsR = 1:100;
98 binsR(binsM) = [];
99 binsR(1:2) = [];
100
101 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
102
103
104 %% mostrar resultados
105 %clc
106 %1 - Taxa de detecção
107 figure
108 plot(TXD, '.k', 'MarkerSize', 10)
109 hold on
110 plot([0 size(TXD, 1)], [TXD(end) TXD(end)], ':r', 'LineWidth', 2)
111 ylabel('Taxa de Detecção', 'fontsize', 12)
112
113 %2 - Falsos Positivo
114 figure
115 plot(FP, '.k', 'MarkerSize', 10)
116 hold on
117 plot([0 size(FP, 1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2)
118 ylabel('Falso Positivo', 'fontsize', 12)
119
120 figure
121 boxplot(FP)
122
123 %%
124 % taxa de detecção x tempo
125 timeM = time(binsM, :);
126 timeM(timeM == -1) = Mmax;
127 timeM = mean(timeM, 1)' * 1; %1segundo por janela
128 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)' * 100;
129
130
131 figure1 = figure; axes1 = axes('Parent', figure1); hold(axes1, 'on');
132 plot([0 1]*100, [Mmax Mmax], '-.k', 'linewidth', 1)
133 plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.k', 'linewidth', 1)
134
135
136 for ii = 1:size(parametros, 1)
```

```

137     plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',[num2str(
(parametros(ii,1)) '-' num2str(parametros(ii,2))])
138 end
139
140 [ p, idxs] = paretoFront([TXD,(-timeM)] );
141 auxL = p(:,1)<0.5;
142 p(auxL,:) = [];
143 idxs(auxL,:) = [];
144
145 [~,ind] = sort(p(:,1));
146 p = p(ind,:);
147 idxs = idxs(ind,:);
148 % p([3,4,7,9],:) = [];
149 % idxs([3,4,7,9],:) = [];
150
151 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
152
153 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
154 % xlim([min(p(:,1))*80 max(p(:,1))*104])
155
156 set(axes1,'XMinorTick','on');
157 set(axes1,'YMinorTick','on');
158 box(axes1,'off');
159 hold off
160 xlabel('Detection Rate (%)','fontsize',12);
161 ylabel('Mean Exam Time (min)','fontsize',12);
162 fprintf('\n');
163
164 for ii = 1:size(idxs,1)
165
166     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
167
168     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
169     fprintf(' NI = %d ',length(I));
170     I = I(1);
171
172     for jj = I
173
174         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros(
(jj,2));
175         %     text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros(
(jj,3)) '\}' ]]);
176 %         text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str(
(parametros(jj,3)) '\}' ]]);
177 %         text(PD_T(jj)*99,T_exam_G(idxs(ii))*0.985,['\{' num2str(
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]]);

```

```
178         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj),✓
1) ' ,' num2str(parametros(jj,2)) '\}' ]);
179     end
180 end
181 fprintf('\n');
182 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
183 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
184
185
186
187
```

```

1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3 %%
4 % caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab';'An';'Bb';'Er';'Lu';...
8     'So';'Qu';'Vi';'Sa';'Ti';'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 % Intensidade -----
13 % intensidade = {'70dB';'60dB';'50dB';'40dB';'30dB';'ESP'}; %quais
intensidade analisadas
14 % sugestão
15 % vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'50dB'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 240; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 %parametros = [Min Mstep Mmax alfa_corrigido]
25 %parametros = [200 1 200 .05];
26 % nRuns = 10000;
27 % Mmax = 240; %número máximo de janela
28 % alfa_teste = 0.05;
29 % FP_desejado =0.05;
30 % [alfa_corrigido,NDC_minimo,cost_alfa, P] =
funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax ,alfa_teste,FP_desejado);
31 alfa = 0.05;
32 FP_desejado = 0.05;
33
34
35 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
36     'NDC_minimo','P', 'nRuns')
37
38 parametros = [P, NDC_minimo,alfa_corrigido];
39
40
41 K_stages = zeros(size(parametros,1),1);
42 for ii = 1:numel(K_stages)
43     K_stages(ii) = numel(parametros(ii,1):parametros(ii,2):parametros

```

```

(ii,3));
44 end
45
46 parametros(K_stages<=5,:) = [];
47 %% Alfa constante
48 % deixar o alfa corrigido em 0.05 nesse caso não há correção
49 for ii=1:size(parametros,1)
50     parametros(ii,5) = 0.05;
51 end
52
53
54 %%
55 load(['eletrodos.mat'])
56 pos_ele = 1;
57
58 ganho = 200;
59 alpha = 0.05;
60
61 %% -----
62 remoc = [.1]/ganho;
63
64
65 %%
66 %*****poder fazer por intensidade aqui -----
67
68 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
69
70     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o ✓
voluntário
71     intensidade = cell2mat(Intensidade); %intensidadde
72     load([voluntario intensidade], 'x','Fs','binsM','freqEstim')
73
74     x = x(:, :, pos_ele);
75
76     nfft = Fs;%1segundo de sinal
77
78     %retirar componente DC por janela (fiz isso pq no processamento em
79     %tempo real é por janela)
80     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada ✓
trecho - devido a remoção
81
82     %scluir os dois primeiros segundos do inicio da coleta
83     x(:,1:2,:) = [];
84
85
86     %encontrar o valor máximo por canal
87     Vmax = max(abs(x), [], 1);
88     ind = Vmax>remoc;

```



```
89     [sum(ind) cont_vol ];
90     x = x(:,~ind); %remover o ruído amplitude
91     x = x(:,1:Mmax);%limitar o tamanho para o valor máximo.
92
93     %***** fazer por canal diferente ----for nCanal = 1:16 %
94     [dr,time] = protocolo_deteccao(x, parametros);
95
96     Tdr(:, :, cont_vol) = dr;
97     Ttime(:, :, cont_vol) = time;
98
99 end
100 % save(['Tdr_50db_M1' '.mat'], 'Tdr');
101 %% Análise de desempenho
102
103 %TXD - analisar as freq. estimulação
104 % binsM = [82      84      86      88      90      92      94      96]
105 %freq. 81Hz,83,85,87,89,91,93,95Hz
106 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
107 binsR = 70:104;
108 binsR(binsM - 71) = [];
109
110 % binsR = binsM+1;
111 % binsR = 1:100;
112 % binsR(1:2) = [];
113 % binsR(60) = [];
114
115 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
116
117 %% FP sob EEG ESP
118 % binsR= binsM;
119 %
120 % FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
121
122 %% mostrar resultados
123 %clc
124 %1 - Taxa de detecção
125
126 figure
127 plot(TXD*100, '.k', 'MarkerSize', 6) % antigo 10 [6]
128 hold on
129 % plot([0 size(TXD,1)], [TXD(end) TXD(end)], ':r', 'LineWidth', 2) %↙
130 %ylabel('Taxa de Detecção', 'fontsize', 12)
131 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
132 ylabel('Taxa de Detecção (%)', 'fontsize', 12)
133 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
134 title(['Estímulo ', (Intensidade{1}), ' SPL: NNDC'])
135 % box off
```

```
136 grid on
137
138 %%
139 %2 - Falsos Positivo
140
141 figure
142 plot(FP*100, '.k', 'MarkerSize', 10)
143 hold on
144 % lsup = 7.41; 95%
145 % linf = 3.03; 95%
146 lsup = 6.73;
147 linf = 3.37;
148
149 plot([0 size(FP,1)], [lsup lsup], ':r', 'LineWidth', 2) %Limite do SUP✓
nível de significancia
150 % plot([0 size(FP,1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2) %✓
Limite do nível de significancia
151 plot([0 size(FP,1)], [linf linf], ':r', 'LineWidth', 2) %Limite do INF✓
nível de significancia
152 plot([0 size(FP,1)], [5 5], ':b', 'LineWidth', 2) %Limite do INF nível de✓
significancia
153
154
155 ylabel('Falso Positivo', 'fontsize', 12)
156 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
157 ylabel('Taxa de Falso Positivo (%)', 'fontsize', 12)
158 box off
159 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
160 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 01'])
161 %figure
162 % boxplot(FP)
163 grid on
164 %%
165
166 % taxa de detecção x tempo
167 timeM = time(binsM, :);
168 timeM(timeM == -1) = Mmax;
169 timeM = mean(timeM, 1) * 1; %1segundo por janela
170 TXD = mean(mean(Tdr(binsM, :, :), 3), 1) * 100;
171
172 %% Single Shot
173 figure1 = figure; axes1 = axes('Parent', figure1); hold(axes1, 'on');
174 plot([0 1]*100, [Mmax Mmax], '-.b', 'linewidth', 1)
175 plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
176
177
178 for ii = 1:size(parametros, 1)
179     plot(TXD(ii), timeM(ii), '.k', 'Markersize', 6, 'DisplayName', [num2str✓
```

```

(parametros(ii,1)) '-' num2str(parametros(ii,2))])
180 end
181
182 [ p, idxs] = paretoFront([TXD, (-timeM)] );
183 auxL = p(:,1)<0.5;
184 p(auxL,:) = [];
185 idxs(auxL,:) = [];
186
187 [~,ind] = sort(p(:,1));
188 p = p(ind,:);
189 idxs = idxs(ind,:);
190 % p([3,4,7,9],:) = [];
191 % idxs([3,4,7,9],:) = [];
192
193 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
194
195 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])
196 % xlim([min(p(:,1))*80 max(p(:,1))*104])
197
198 set(axes1,'XMinorTick','on');
199 set(axes1,'YMinorTick','on');
200 box(axes1,'off');
201 hold off
202 xlabel('Taxa de Detecção (%)','fontsize',12);
203 ylabel('Tempo Médio de Exame (s)','fontsize',12);
204 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
205 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 01'])
206
207 fprintf('\n');
208
209 for ii = 1:size(idxs,1)
210
211     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
212
213     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
214     fprintf(' NI = %d ',length(I));
215     I = I(1);
216
217     for jj = I
218
219         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
220         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
221         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);

```

```
222 %         text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str(↵
(parameters(jj,1)) ',' num2str(parameters(jj,2)) ',100\}' ]);
223         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parameters(jj,↵
1)) ',' num2str(parameters(jj,2)) '\}' ]);
224     end
225 end
226 fprintf('\n');
227 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
228 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
229
230 grid on
231 %% Salvar
232 % save(['timeM_30db_M1' '.mat'],'timeM')
233 % save(['pareto_30db_M1' '.mat'],'TXD','Mmax','parameters');
234
235 % save(['timeM_50db_M1' '.mat'],'timeM')
236 % save(['pareto_50db_M1' '.mat'],'TXD','Mmax','parameters');
237
238 % save(['Workspace_M1' '.mat']); %50dB
239 % save(['Workspace_M1_30' '.mat']); %30dB
240
```

```
1 %Aplicação do protocolo ao banco de dados
2 clear all, close all, clc
3
4 % caminho = 'C:\
\Users\Crohma\Documents\PNV\Simulation\Testes_Seq\Sinais_EEG\';
5
6 %vetor dos voluntários
7 Vvoluntario = {'Ab';'An';'Bb';'Er';'Lu';...
8     'So';'Qu';'Vi';'Sa';'Ti';'Wr'}; %vetor dos voluntário
9
10 % Vvoluntario = Vvoluntario([1,2]);
11
12 %Intensidade -----
13 %intensidade = {'70dB';'60dB';'50dB';'40dB';'30dB';'ESP'}; %quais
intensidade analisadas
14 %sugestão
15 %vetor_Mmax = [50;50;240;440;440;20]; %número máximo de janela para
cada intensidade
16 Intensidade = {'50dB'};
17 % Intensidade = {'ESP'};
18 % Intensidade = {'60dB'};
19
20 Mmax = 240; %valor máximo
21
22 %% Parametros do protocolo de detecção.
23
24 alfa = 0.05;
25 FP_desejado = 0.05;
26 % Dados gerados por Colatina montar matrix parametros ' [P, NDC_minimo,
alfa_corrigido]'
27
28 load(['NDC_AlfaCorrigido_Mmax' num2str(Mmax) '_alfa_' num2str(alfa)
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
29     'NDC_minimo','P', 'nRuns')
30 parametros = [P, NDC_minimo,alfa_corrigido];
31 %% Alfa constante
32 % for ii=1:size(parametros,1)
33 %     parametros(ii,5) = 0.05;
34 % end
35
36
37 %%
38 load(['eletrodos.mat'])
39 pos_ele = 1; %
40
41 ganho = 200;
42 alpha = 0.05;
43
```

```

44 %% -----
45 remoc = [.1]/ganho;
46 %%
47 %*****poder fazer por intensidade aqui -----
48
49 for cont_vol = 1:size(Vvoluntario,1) %fazer por voluntário
50
51     voluntario = cell2mat(Vvoluntario(cont_vol,:)); %carregar o ✓
voluntário
52     intensidade = cell2mat(Intensidade); %intensidadde
53     load([voluntario intensidade], 'x','Fs','binsM','freqEstim')
54
55     x = x(:, :, pos_ele);
56
57     nfft = Fs;%1segundo de sinal
58
59     %retirar componente DC por janela (fiz isso pq no processamento em
60     %tempo real é por janela)
61     x = x - repmat(mean(x),nfft,1); %tirar a média de cada de cada ✓
trecho - devido a remoção
62
63     %scluir os dois primeiros segundos do inicio da coleta
64     x(:,1:2,:) =[];
65
66
67     %encontrar o valor máximo por canal
68     Vmax = max(abs(x), [], 1);
69     ind = Vmax>remoc;
70     [sum(ind) cont_vol ];
71     x = x(:, ~ind); %remover o ruído amplitude
72     x = x(:, 1:Mmax);%limitar o tamanho para o valor máximo.
73
74     %***** fazer por canal diferente ----for nCanal = 1:16 %
75     [dr,time] = protocolo_deteccao(x, parametros);
76
77     Tdr(:, :, cont_vol) = dr;
78     Ttime(:, :, cont_vol) = time;
79
80 end
81 %save(['Tdr_50db_M2' '.mat'], 'Tdr');
82 %% Análise de desempenho
83
84 %TXD - analisar as freq. estimulação
85 % binsM = [82      84      86      88      90      92      94      96 ];
86 % binsM2 = [binsM 2*binsM 3*binsM];
87
88 %%
89 %freq. 81Hz,83,85,87,89,91,93,95Hz

```

```
90 TXD = mean(mean(Tdr(binsM, :, :), 3), 1)';
91 %%
92 % binsR = binsM+1;
93 % binsR = 1:100;
94 % binsR(binsM) = [];
95 % binsR(1:2) = [];
96
97
98 binsR = 70:104;
99 binsR(binsM - 71) = [];
100
101 FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
102 %% FP sob EEG ESP
103 % binsR= binsM;
104 %
105 % FP = mean(mean(Tdr(binsR, :, :), 3), 1)';
106
107 %% mostrar resultados
108 %clc
109 %1 - Taxa de detecção
110 figure
111 plot(TXD, '.k', 'MarkerSize', 6) % antigo 10 [6]
112 hold on
113 plot([0 size(TXD, 1)], [TXD(1) TXD(1)], ':r', 'LineWidth', 2) %single shot✓
first line
114 %ylabel('Taxa de Detecção', 'fontsize', 12)
115 xlabel('Índice dos conjuntos de Parâmetros', 'fontsize', 12)
116 ylabel('Taxa de Detecção (%)', 'fontsize', 12)
117 % title([ (Intensidade{1}), 'ONTÂNEO: Método 04'])
118 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
119 % box off
120 grid on
121
122 %%
123 %2 - Falsos Positivo
124 figure
125 plot(FP*100, '.k', 'MarkerSize', 10)
126 hold on
127 % lsup = 7.41; 95%
128 % linf = 3.03; 95%
129 lsup = 6.73;
130 linf = 3.37;
131
132 plot([0 size(FP, 1)], [lsup lsup ], ':r', 'LineWidth', 2) %Limite do SUP✓
nivel de significancia
133 % plot([0 size(FP, 1)], [FP_desejado FP_desejado], ':r', 'LineWidth', 2) %✓
Limite do nivel de significancia
134 plot([0 size(FP, 1)], [linf linf], ':r', 'LineWidth', 2) %Limite do INF✓
```

```

nivel de significancia
135
136 %ylabel('Falso Positivo','fontsize',12)
137 xlabel('Índice dos conjuntos de Parâmetros','fontsize',12)
138 ylabel('Taxa de Falso Positivo (%)','fontsize',12)
139 box off
140 % title([ (Intensidade{1}), 'ONTÂNEO: Método 02'])
141 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
142 %figure
143 ylim([0 inf])
144
145 % boxplot(FP)
146 grid on
147 %%
148 % taxa de detecção x tempo
149 timeM = time(binsM,:);
150 timeM(timeM==-1) = Mmax;
151 timeM = mean(timeM,1)*1; %1segundo por janela
152 TXD = mean(mean(Tdr(binsM, :, :), 3), 1) * 100;
153
154 %% Single Shot
155 figure1 = figure; axes1 = axes('Parent',figure1); hold(axes1,'on');
156 plot([0 1]*100, [Mmax Mmax], '-.b', 'linewidth', 1)
157 plot([TXD(1) TXD(1)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
158 %Banco de dados com single shot na última linha
159 % plot([TXD(end) TXD(end)], [min(timeM) max(timeM)], '-.b', 'linewidth', 1)
160
161
162 for ii = 1:size(parametros,1)
163     plot(TXD(ii),timeM(ii),'.k','Markersize',6,'DisplayName',[num2str(
parametros(ii,1)) '-' num2str(parametros(ii,2))])
164 end
165
166 [ p, idxs] = paretoFront([TXD, (-timeM)] );
167 auxL = p(:,1)<0.5;
168 p(auxL,:) = [];
169 idxs(auxL,:) = [];
170
171 [~,ind] = sort(p(:,1));
172 p = p(ind,:);
173 idxs = idxs(ind,:);
174 % p([3,4,7,9],:) = [];
175 % idxs([3,4,7,9],:) = [];
176
177 plot(TXD(idxs),timeM(idxs),'-or','Markersize',8,'linewidth',1.2)
178
179 % ylim([min(-p(:,2))*0.9 max(-p(:,2))*1.1])

```



```

180 % xlim([min(p(:,1))*80 max(p(:,1))*104])
181
182 set(axes1,'XMinorTick','on');
183 set(axes1,'YMinorTick','on');
184 box(axes1,'off');
185 hold off
186 xlabel('Taxa de Detecção (%)','fontsize',12);
187 ylabel('Tempo Médio de Exame (s)','fontsize',12);
188 % title([ (Intensidade{1}), 'ONTÂNEO: Método 02'])
189 title(['Estímulo ', (Intensidade{1}), ' SPL: Método 02'])
190
191 fprintf('\n');
192
193 for ii = 1:size(idxs,1)
194
195     [I] = find((TXD == TXD(idxs(ii))) & (timeM==timeM(idxs(ii))));
196
197     fprintf('\nPD = %f Tempo = %f ',TXD(idxs(ii)),timeM(idxs(ii)));
198     fprintf(' NI = %d ',length(I));
199     I = I(1);
200
201     for jj = I
202
203         fprintf(' - Buffer:%d, M_step:%d', parametros(jj,1),parametros
(jj,2));
204         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',' num2str(parametros
(jj,3)) '\}' ]);
205         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100,' num2str
(parametros(jj,3)) '\}' ]);
206         % text(PD_T(jj)*99,T_exame_G(idxs(ii))*0.985,['\{' num2str
(parametros(jj,1)) ',' num2str(parametros(jj,2)) ',100\}' ]);
207         text(TXD(jj),timeM(idxs(ii))*0.975,['\{' num2str(parametros(jj,
1)) ',' num2str(parametros(jj,2)) '\}' ]);
208     end
209 end
210 fprintf('\n');
211 xlim([min(TXD(idxs))*0.95,max(TXD(idxs))*1.05])
212 ylim([min(timeM(idxs))*0.95 Mmax*1.05])
213
214 grid on
215 save(['timeM_50db_M1' '.mat'],'timeM')
216 save(['pareto_50db_M1' '.mat'],'TXD','Mmax','parametros');
217 %% save(['Workspace_M2' '.mat']); %50dB
218 % save(['Workspace_M2_30' '.mat']); %30dB
219

```

```
1 % clear all
2 % close all
3 % clc
4
5 nRuns = 10000;
6 Mmax = 440; %número máximo de janela
7 alfa = 0.05;
8 FP_desejado =0.05;
9
10
11 [alfa_corrigido,NDC_minimo,cost_alfa, P] = funcao_NDC_alfaCorrigido_Mmax✓
(nRuns,Mmax ,alfa,FP_desejado);
12 %salvar as variáveis
13 save(['NDC_AlfaCorrigido_Mmax_Col' num2str(Mmax) '_alfa_' num2str(alfa)✓
'_FPdesejado' num2str(FP_desejado) '.mat'],'alfa_corrigido', ...
14      'NDC_minimo','P', 'nRuns','alfa','FP_desejado')
15
16
17 t1 = table(P, NDC_minimo,alfa_corrigido,cost_alfa);
18
19 %% Avaliar a variância na estimação do NDC e alfa
20
21 %repetir isso várias vezes
22 % [alfa_corrigido,NDC_minimo,cost_alfa, P] =✓
funcao_NDC_alfaCorrigido_Mmax(nRuns,Mmax ,alfa_teste,FP_desejado);
23 % comparar os valor estimados e calcular a variância
24
25 %%
26
27
28
29
```